



INSTITUTO POLITECNICO NACIONAL “IPN”



ESCUELA SUPERIOR DE COMPUTO “ESCOM”

LANDEROS CORTES PEDRO JONAS

SISTEMAS DISTRIBUIDOS

TAREA 8

2023630402

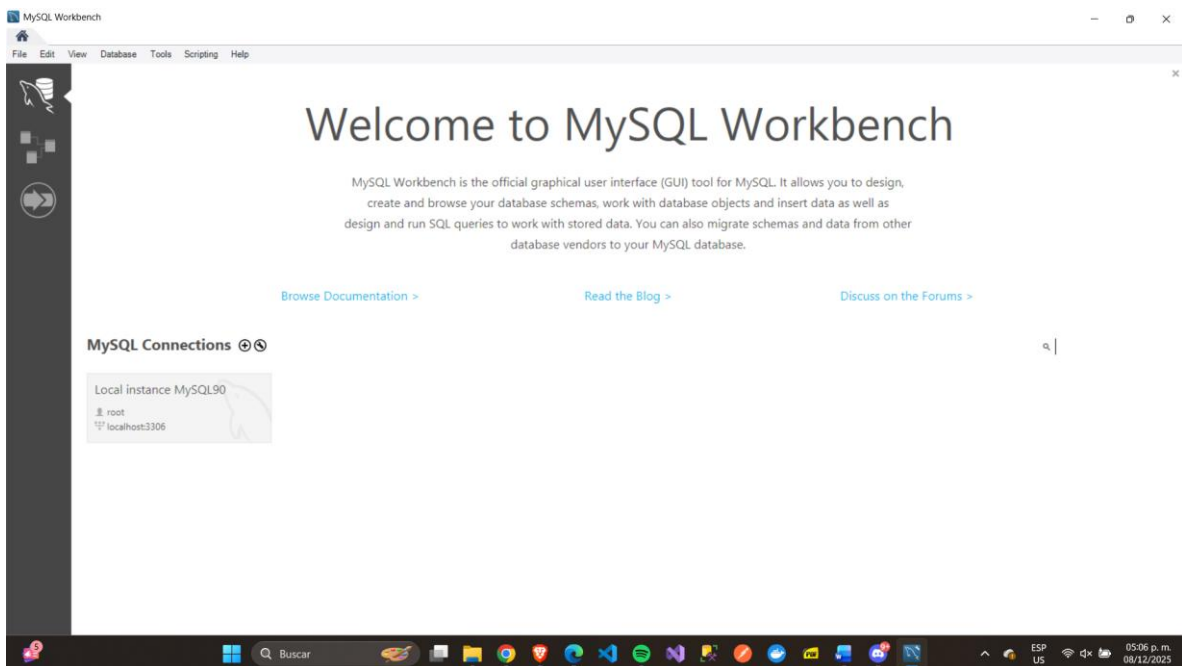
Indice

MYSQL Local	3
Creación de base de datos y tablas	4
Crear un usuario local para la base de datos	4
Backend	5
Creación de la carpeta con la nomenclatura	5
Instalacion de extensiones	5
Creacion del proyecto	6
Instalacion de paquetes	7
Modificación de localsettings	8
Compilación de proyecto	8
Creacion de las tablas stock, fotos_articulos y carrito_compra	9
Creacion de funciones en el backed.	11
Alta_articulo	11
consulta_articulos	12
compra_articulo	13
elimina_articulo_carrito_compra	15
elimina_carrito_compra	16
modifica_carrito_compra	17
Pruebas backend local	19
Creacion de la function app	22
Deploy de la app	24
Cuenta de almacenamiento.....	25
Recurso compartido de archivos.....	26
Creacion.....	26
Subida de archivos	27
Montaje de almacenamiento en la aplicación de funciones	28
Configuración de variables de entorno en aplicación de funciones	29
Root.....	29

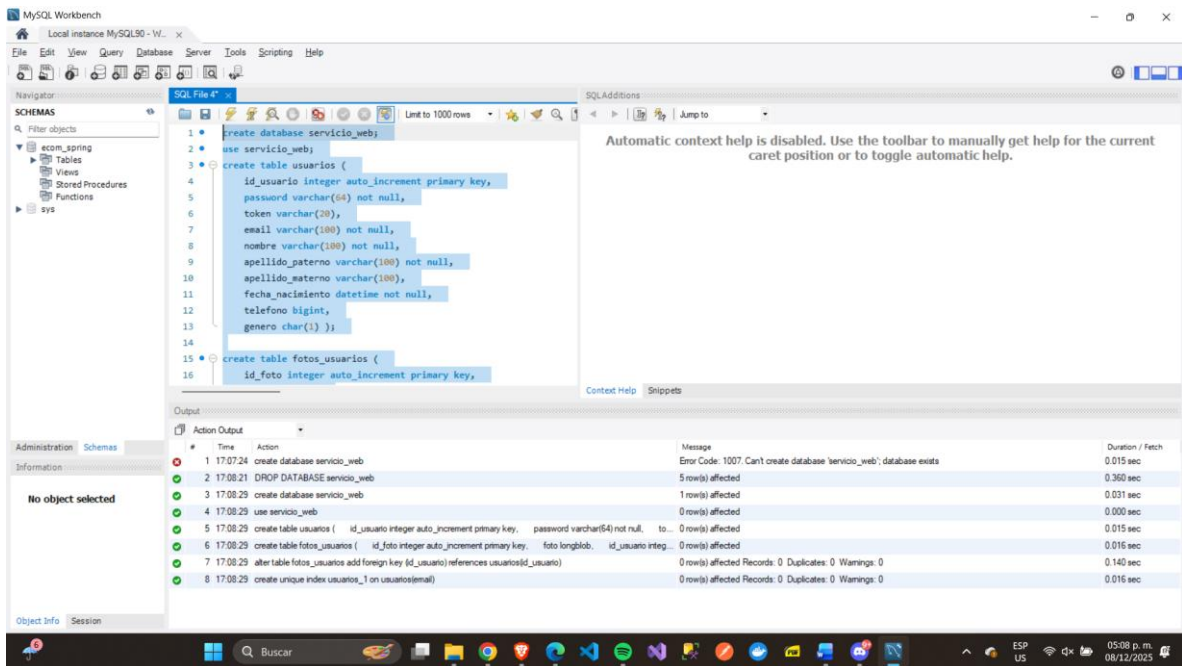
Server	29
Creacion de base de datos PaaS	30
Pruebas de la app desde el celular	35
Pruebas de la function app desde Postman.....	51
Conclusiones	55

MYSQL Local

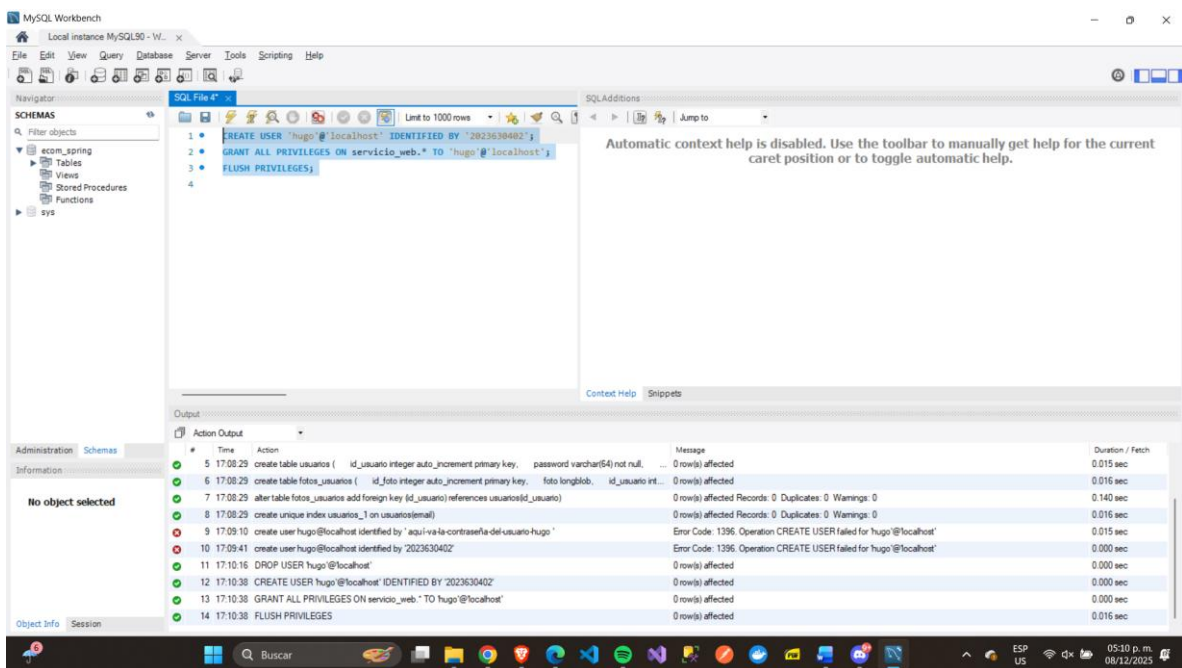
Ya se tenía una instalación previamente, por lo que ya no fue necesario volverlo a instalar



Creación de base de datos y tablas

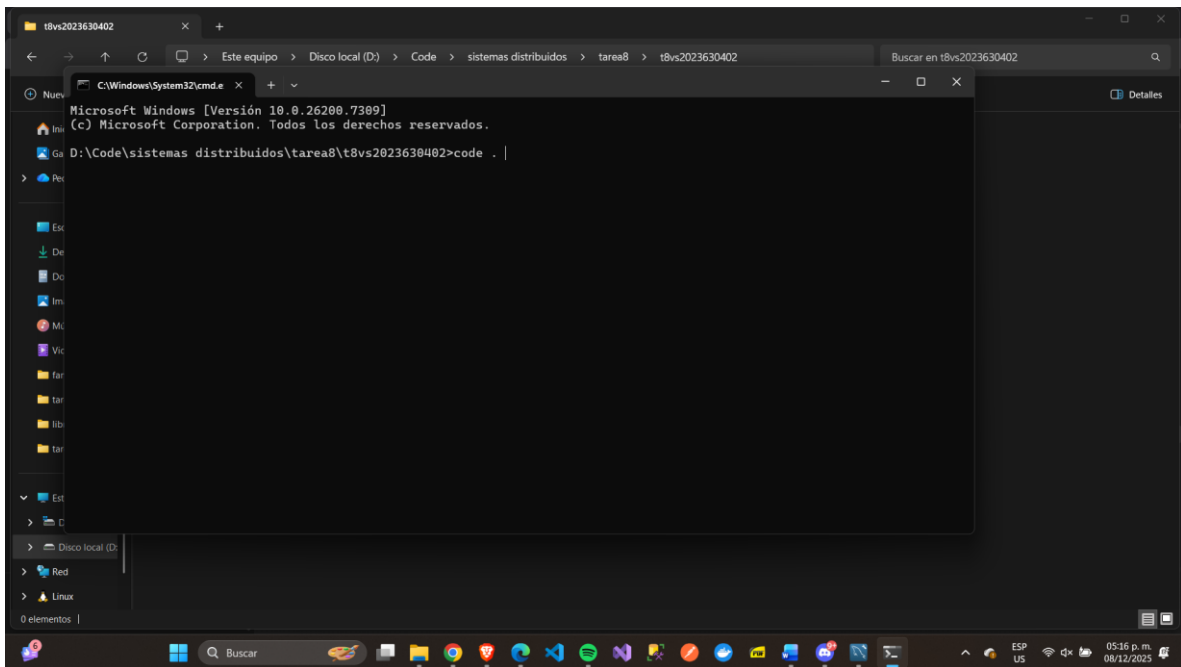


Crear un usuario local para la base de datos

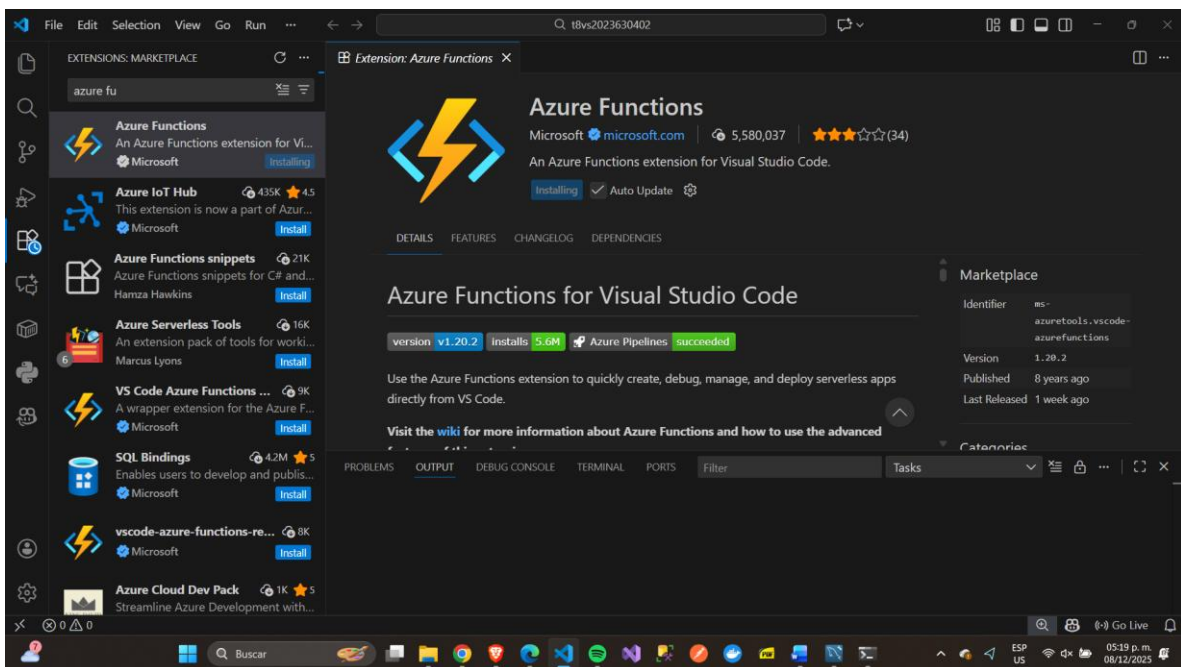


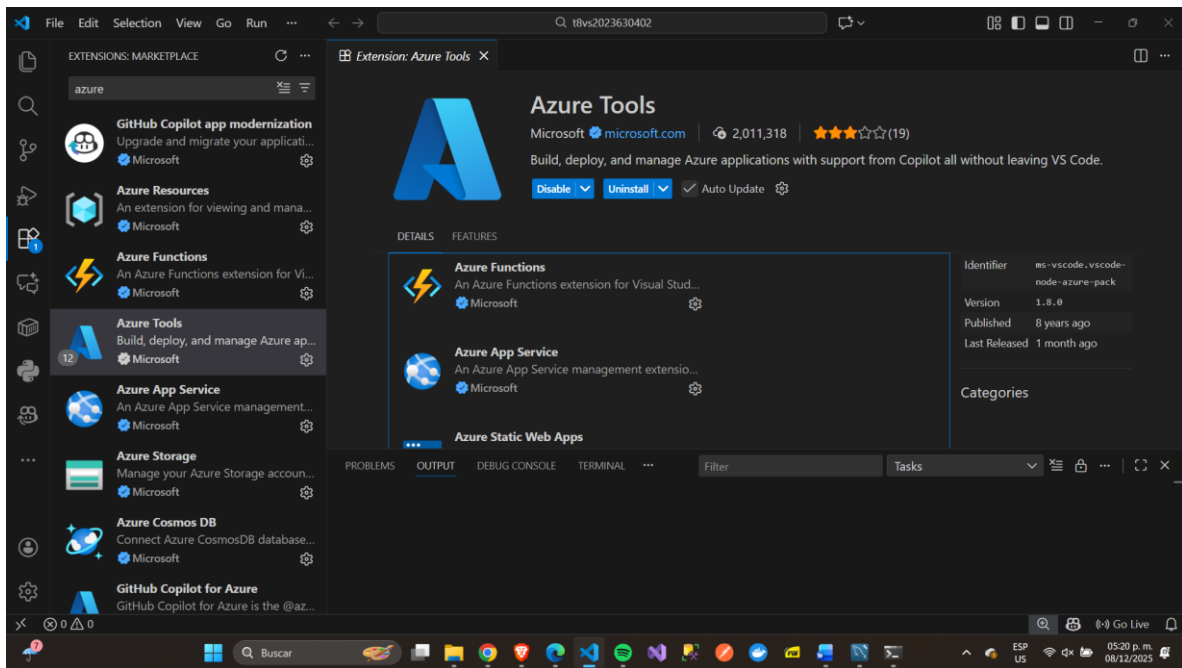
Backend

Creación de la carpeta con la nomenclatura

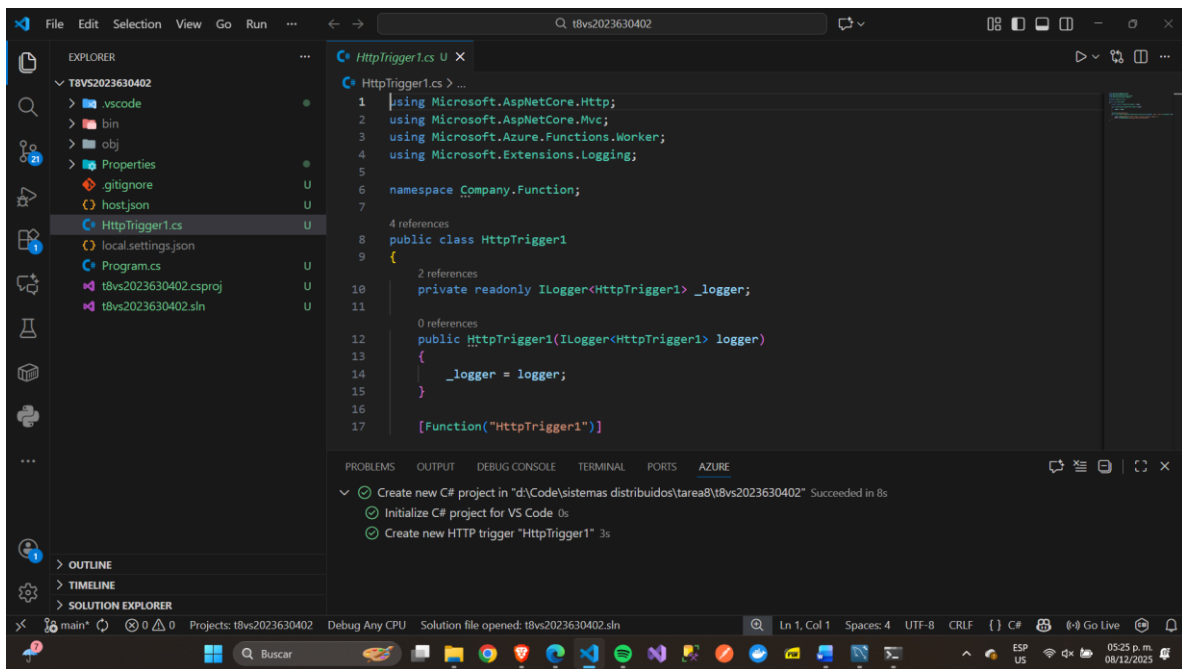


Instalacion de extensiones





Creacion del proyecto



Instalacion de paquetes

This screenshot shows the Visual Studio Code interface with a C# project named 'login.cs'. The Explorer pane on the left shows the project structure, including files like 'login.cs', 'MimeMapping.cs', and 'Program.cs'. The main editor displays the 'login.cs' file with the following code:

```
1 // (c) Carlos Pineda Guerrero. 2025
2 using Microsoft.AspNetCore.Http;
3 using Microsoft.AspNetCore.Mvc;
4 using Microsoft.Azure.Functions.Worker;
5 using Microsoft.Extensions.Logging;
6 using Newtonsoft.Json;
7 using MySql.Data.MySqlClient;
8 using System.Security.Cryptography;
9 using System.Text;
10
11 namespace servicio;
12
13 4 references
14 public class login
15 {
16     2 references
17     class HuboError
18     {
19         1 reference
20         public string mensaje;
```

The TERMINAL pane at the bottom shows the command 'dotnet add package Newtonsoft.Json' being executed. The output indicates that the package was successfully added to the project.

```
PS D:\Code\sistemas distribuidos\tarea8\t8vs2023630402> dotnet add package Newtonsoft.Json
.NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por
.NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "Newtonsoft.Json" al proyecto "D:\Code\sistemas distribuidos\tarea8\t8vs
2023630402\t8vs2023630402.csproj".
info : GET https://api.nuget.org/v3/registration5-gz-semver2/newtonsoft.json/index.json
```

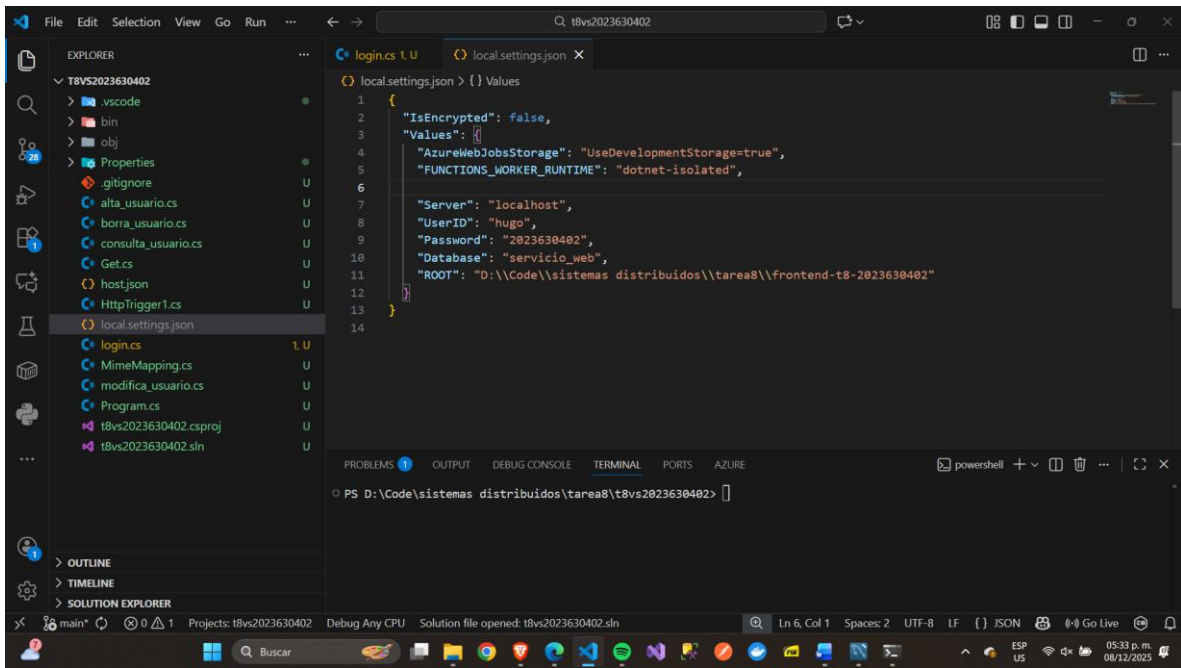
This screenshot shows the Visual Studio Code interface with the same C# project. The Explorer pane on the left shows the project structure, including files like 'login.cs', 'MimeMapping.cs', and 'Program.cs'. The main editor displays the 'login.cs' file with the following code:

```
1 // (c) Carlos Pineda Guerrero. 2025
2 using Microsoft.AspNetCore.Http;
3 using Microsoft.AspNetCore.Mvc;
4 using Microsoft.Azure.Functions.Worker;
5 using Microsoft.Extensions.Logging;
6 using Newtonsoft.Json;
7 using MySql.Data.MySqlClient;
8 using System.Security.Cryptography;
9 using System.Text;
10
11 namespace servicio;
12
13 4 references
14 public class login
15 {
16     2 references
17     class HuboError
18     {
19         1 reference
20         public string mensaje;
```

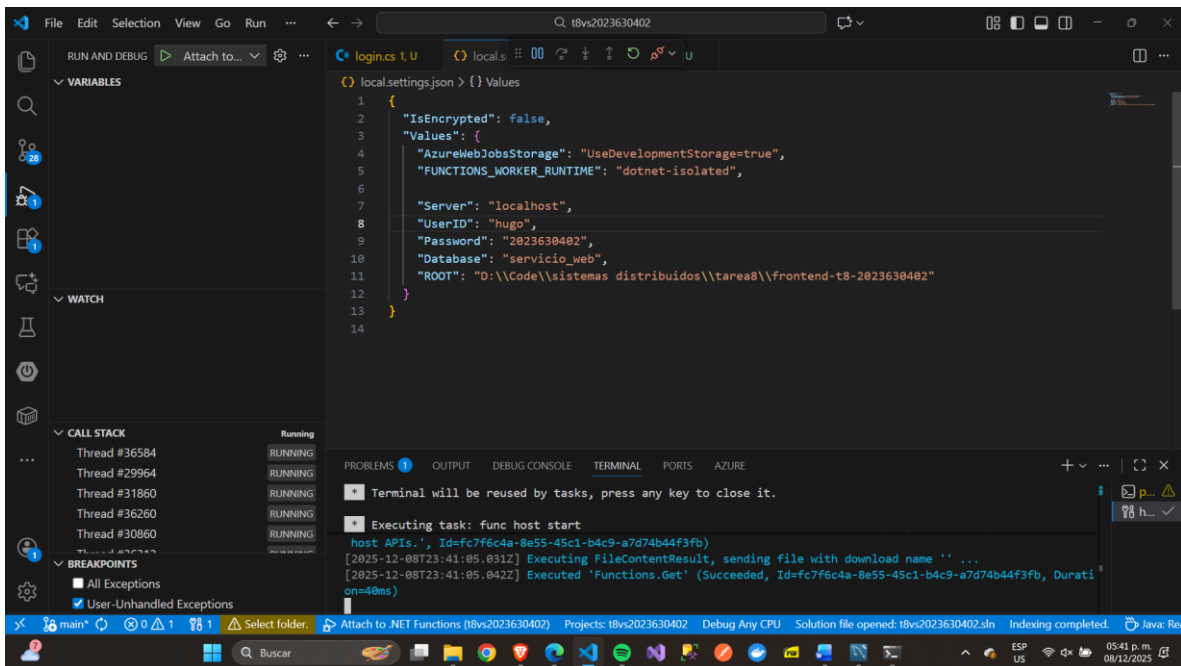
The TERMINAL pane at the bottom shows the command 'dotnet add package MySQL.Data' being executed. The output indicates that the package was successfully added to the project.

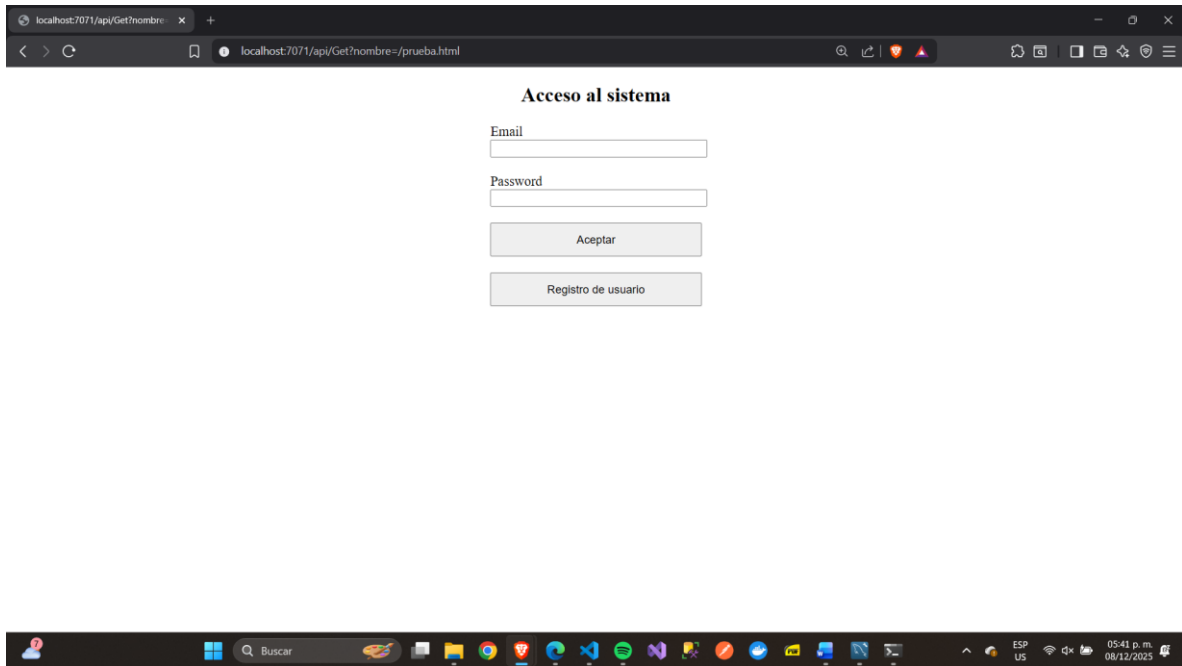
```
PS D:\Code\sistemas distribuidos\tarea8\t8vs2023630402> dotnet add package MySQL.Data
distribuidos\tarea8\t8vs2023630402\t8vs2023630402.csproj".
info : Se agregó PackageReference para la versión "9.5.0" del paquete "MySQL.Data" al archivo "D:\Code\sistemas distribuid
os\tarea8\t8vs2023630402\t8vs2023630402.csproj".
info : Escribiendo el archivo de recursos en el disco. Ruta de acceso: D:\Code\sistemas distribuidos\tarea8\t8vs2023630402
\obj\project.assets.json
log : Se ha restaurado D:\Code\sistemas distribuidos\tarea8\t8vs2023630402\t8vs2023630402.csproj (en 3.82 s).
```

Modificación de localsettings



Compilación de proyecto





Creacion de las tablas stock, fotos_articulos y carrito_compra

Script utilizado:

USE servicio_web;

-- 1) Tabla de artículos en stock

CREATE TABLE stock (

id_articulo INT AUTO_INCREMENT PRIMARY KEY,

nombre VARCHAR(100) NOT NULL,

descripcion VARCHAR(255) NOT NULL,

precio DECIMAL(10,2) NOT NULL,

existencia INT NOT NULL,

id_usuario INT NOT NULL,

FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)

```
);
```

```
-- 2) Tabla de fotos de artículos
```

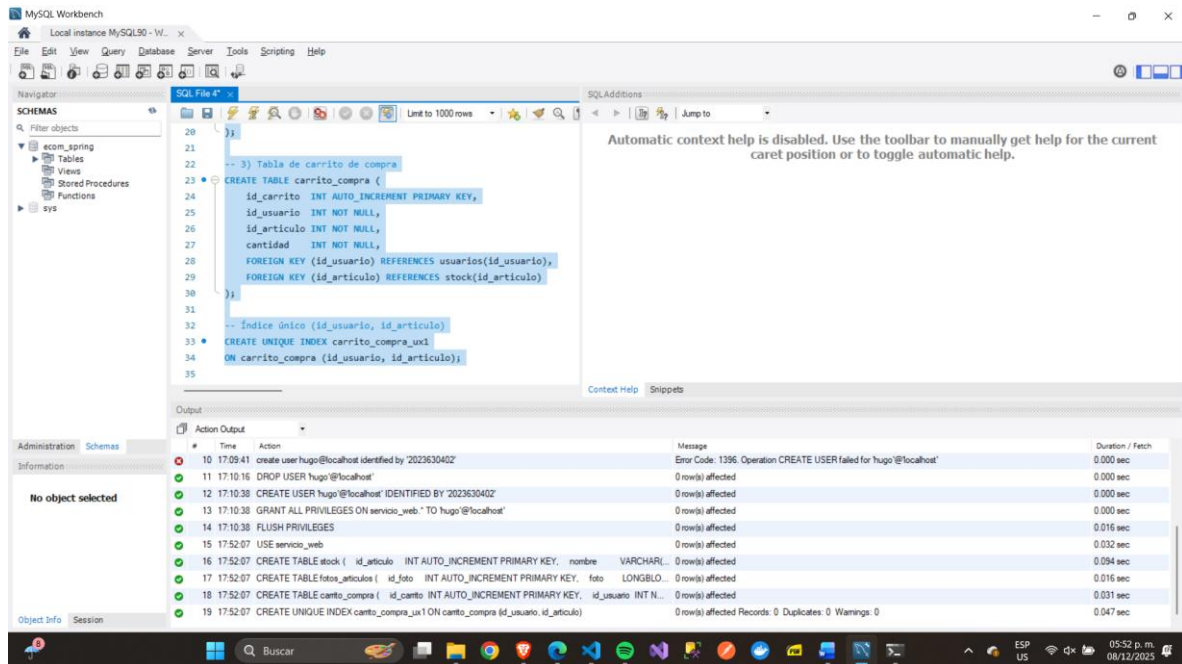
```
CREATE TABLE fotos_articulos (  
    id_foto INT AUTO_INCREMENT PRIMARY KEY,  
    foto LONGBLOB NOT NULL,  
    id_articulo INT NOT NULL,  
    FOREIGN KEY (id_articulo) REFERENCES stock(id_articulo)  
);
```

```
-- 3) Tabla de carrito de compra
```

```
CREATE TABLE carrito_compra (  
    id_carrito INT AUTO_INCREMENT PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    id_articulo INT NOT NULL,  
    cantidad INT NOT NULL,  
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),  
    FOREIGN KEY (id_articulo) REFERENCES stock(id_articulo)  
);
```

```
-- Índice único (id_usuario, id_articulo)
```

```
CREATE UNIQUE INDEX carrito_compra_ux1  
ON carrito_compra (id_usuario, id_articulo);
```



Creacion de funciones en el backed.

NOTA: Las funciones completas están adjuntas en el repositorio y en la entrega de la tarea.

Alta_articulo

Este código implementa una **Azure Function** llamada **alta_articulo**, cuya finalidad es **dar de alta un artículo** en la base de datos.

Entrada de datos

Se define la clase **Articulo** para recibir el cuerpo de la petición en formato JSON, el cual incluye:

- **id_usuario** y **token** para validación de acceso
- **nombre**, **descripcion**, **precio** y **cantidad** del artículo
- **foto** opcional codificada en **Base64**

Funcionamiento general

1. Se **lee y deserializa** el cuerpo de la petición.

2. Se **validan los campos obligatorios**, asegurando que los datos sean correctos y consistentes.
3. Se obtiene la **cadena de conexión** a MySQL a partir de variables de entorno.
4. Se abre la conexión y se inicia una **transacción** para mantener la integridad de los datos.

Validación de acceso

Antes de realizar inserciones, se valida que el `id_usuario` y el token correspondan a un usuario autorizado mediante el método `login.verifica_acceso`.

Operaciones en la base de datos

- Se inserta el artículo en la tabla **stock**.
- Se recupera el `id_articulo` generado automáticamente.
- Si se proporciona una imagen, esta se convierte de Base64 a bytes y se guarda en la tabla **fotos_articulos**.

Manejo de errores

- Si el proceso se completa correctamente, se confirma la transacción y se devuelve un mensaje de éxito.
- Si ocurre un error, la transacción se revierte y se responde con un mensaje de error en formato JSON.

consulta_articulos

Este código implementa una **Azure Function** llamada **consulta_articulos** que permite **buscar artículos** en la base de datos usando una palabra clave y regresar la información (incluida la foto si existe).

Parámetros que recibe (query string)

Se leen desde la URL:

- `palabra`: texto a buscar
- `id_usuario` y `token`: para validar acceso

Si falta alguno, se regresa error.

Flujo general

1. **Valida** que palabra, id_usuario y token estén presentes.
2. Obtiene credenciales de MySQL desde **variables de entorno** y abre conexión.
3. Verifica el acceso con login.verifica_acceso(conexion, id_usuario, token).
 - Si falla, devuelve **“Acceso denegado”**.

Consulta SQL

Hace un SELECT a la tabla stock y une (si existe) la imagen desde fotos_articulos con un:

- LEFT OUTER JOIN (permite traer artículos aunque no tengan foto)

Filtra con LIKE para buscar la palabra en:

- nombre o descripcion

Construcción de la respuesta

- Recorre el SqlDataReader y llena una lista de Articulo.
- Si existe foto (IsDBNull), la lee como bytes (GetBytes) y la convierte a **Base64** para mandarla en JSON.
- Devuelve **200 OK** con la lista serializada en JSON.

Manejo de errores

Si ocurre cualquier excepción, responde **400 BadRequest** con un JSON:

```
{ "mensaje": "..." }
```

compra_articulo

Este código implementa una **Azure Function** llamada **compra_articulo** que permite **agregar un artículo al carrito** y **descontar el stock** en la base de datos, asegurando consistencia con una transacción.

Entrada de datos (JSON)

Se deserializa una Peticion con:

- id_usuario y token (validación de acceso)

- id_articulo (artículo a comprar)
- cantidad (unidades a agregar)

Flujo general

1. **Lee el body** y lo convierte a Peticion.
2. **Valida** que existan id_usuario, token, id_articulo y que cantidad > 0.
3. Abre conexión a MySQL (con variables de entorno) e inicia una **transacción (tx)**.

Validación de acceso

Se verifica que el usuario esté autorizado con:

- login.verifica_acceso(conexion, id_usuario, token)
Si falla, lanza **“Acceso denegado”**.

Operaciones dentro de la transacción

1. **Verificar stock disponible**
 - Consulta SELECT cantidad FROM stock WHERE id_articulo = ...
 - Si no existe el artículo → error
 - Si la cantidad solicitada es mayor al stock → **“Stock insuficiente”**
2. **Insertar o actualizar carrito**
 - Inserta en carrito_compra
 - Si ya existía el registro (misma combinación de usuario/artículo), usa ON DUPLICATE KEY UPDATE para **sumar** la cantidad nueva a la existente.
3. **Actualizar stock**
 - Resta la cantidad comprada en stock con:
UPDATE stock SET cantidad = cantidad - @cantidad

Manejo de errores

- Si todo sale bien: Commit() y responde **200 OK** con *“Artículo agregado al carrito”*.

- Si ocurre un error: Rollback() y responde **400 BadRequest** con JSON { "mensaje": "..."}.

elimina_articulo_carrito_compra

Este código implementa una **Azure Function** llamada **elimina_articulo_carrito_compra** que permite **eliminar un artículo del carrito y devolver su cantidad al stock**, usando una transacción para mantener consistencia.

Parámetros que recibe (query string)

- id_usuario
- token
- id_articulo

Si falta alguno, se devuelve error.

Flujo general

1. **Lee y valida** los parámetros de la URL.
2. Convierte id_usuario e id_articulo a enteros.
3. Abre conexión a MySQL e inicia una **transacción (tx)**.

Validación de acceso

Se valida que el usuario tenga permiso con:

- login.verifica_acceso(conexion, id_usuario, token)
Si falla, se lanza **“Acceso denegado”**.

Operaciones dentro de la transacción

1. **Consultar cantidad en el carrito**
 - Busca cantidad en carrito_compra para ese usuario y artículo.
 - Si no existe registro → **“El artículo no existe en el carrito”**.
2. **Regresar unidades al stock**
 - Actualiza stock sumando esa cantidad:
cantidad = cantidad + cantCarrito

3. Eliminar el registro del carrito

- Hace DELETE en carrito_compra para ese usuario y artículo.

Manejo de errores

- Si todo sale bien: Commit() y responde **200 OK** con “*Artículo eliminado del carrito*”.
- Si ocurre un error: Rollback() y responde **400 BadRequest** con JSON { "mensaje": "..." }.

elimina_carrito_compra

Este código implementa una **Azure Function** llamada **elimina_carrito_compra** que permite **vaciar por completo el carrito de un usuario**, devolviendo primero todas las cantidades al inventario, todo dentro de una transacción.

Parámetros que recibe (query string)

- id_usuario
 - token
- Si falta alguno, devuelve error.

Flujo general

1. **Lee y valida** id_usuario y token.
2. Abre conexión a MySQL (con variables de entorno) e inicia una **transacción (tx)**.
3. Verifica el acceso con login.verifica_acceso(conexion, id_usuario, token).

Operaciones dentro de la transacción

1. **Obtener todos los artículos del carrito**
 - Consulta carrito_compra filtrando por id_usuario.
 - Guarda cada registro como (id_articulo, cantidad) en una lista.
2. **Regresar existencias al stock**

- Recorre la lista y para cada artículo actualiza stock sumando la cantidad:
 $\text{cantidad} = \text{cantidad} + @\text{cantidad}$

3. Eliminar el carrito

- Borra todos los registros del usuario en carrito_compra con un DELETE.

Manejo de errores

- Si todo sale bien: Commit() y responde **200 OK** con “Se eliminó el carrito”.
- Si ocurre un error: Rollback() y responde **400 BadRequest** con JSON {
"mensaje": "..."}.

modifica_carrito_compra

Este código implementa una **Azure Function** llamada **modifica_carrito_compra** que permite **aumentar o disminuir** la cantidad de un artículo en el carrito usando un campo delta, y al mismo tiempo **actualiza el stock** de forma consistente mediante una transacción.

Entrada de datos (JSON)

Se recibe una Peticion con:

- id_usuario y token (validación de acceso)
- id_articulo (artículo a modificar)
- delta (cambio en cantidad): **+1** incrementa, **-1** decrementa (no se permite 0)

Flujo general

1. Lee el body y deserializa el JSON.
2. Valida que existan id_usuario, token, id_articulo y que delta != 0.
3. Abre conexión MySQL e inicia una **transacción (tx)**.
4. Valida acceso con login.verifica_acceso.

Operación dentro de la transacción

1. **Obtiene la cantidad actual en el carrito**
 - Si no hay registro, se considera cantActual = 0.

- Si se intenta decrementar ($\text{delta} < 0$) y $\text{cantActual} == 0$, se lanza error (no existe en carrito).

2. Si $\text{delta} > 0$ (incrementar)

- Consulta el stock en stock y valida:
 - que el artículo exista
 - que haya disponibilidad suficiente para incrementar
- Si el artículo no estaba en el carrito ($\text{cantActual} == 0$), lo **inserta**.
- Si ya estaba, hace **UPDATE** sumando delta.
- Finalmente, descuenta del inventario:
 - $\text{stock.cantidad} = \text{stock.cantidad} - \text{delta}$

3. Si $\text{delta} < 0$ (decrementar)

- Calcula $\text{nuevaCantidad} = \text{cantActual} + \text{delta}$ (delta es negativo).
- Devuelve al inventario -delta unidades:
 - $\text{stock.cantidad} = \text{stock.cantidad} + \text{devolver}$
- Si $\text{nuevaCantidad} \leq 0$, elimina el registro del carrito (DELETE).
- Si todavía queda cantidad, actualiza el carrito dejando la cantidad exacta (UPDATE cantidad = nuevaCantidad).

Manejo de errores

- Si todo sale bien: Commit() y responde **200 OK** con *"Carrito modificado"*.
- Si ocurre un error: Rollback() y responde **400 BadRequest** con JSON {
"mensaje": "..."}.

Pruebas backend local

The screenshot shows the Postman interface with a workspace named 'API Network'. The left sidebar displays a collection of API endpoints under 'Servicio - Backend Sistemas Distribuidos / Usuarios'. The main panel shows a POST request to 'POST 1 - A' with the URL 'http://localhost:7071/api/alta_usuario'. The request body is a JSON object containing user details. The response is a 200 OK status with a message: 'Se dio de alta el usuario'.

```
POST http://localhost:7071/api/alta_usuario
```

```
{
  "password": "123456789012",
  "nombre": "Pedro",
  "apellido_paterno": "Landeros",
  "apellido_materno": "Cortes",
  "fecha_nacimiento": "2000-01-15T00:00:00.000",
  "telefono": "5512345678",
  "genero": "M"
}
```

Response: 200 OK · 1.18 s · 181 B

```
{
  "mensaje": "Se dio de alta el usuario"
}
```

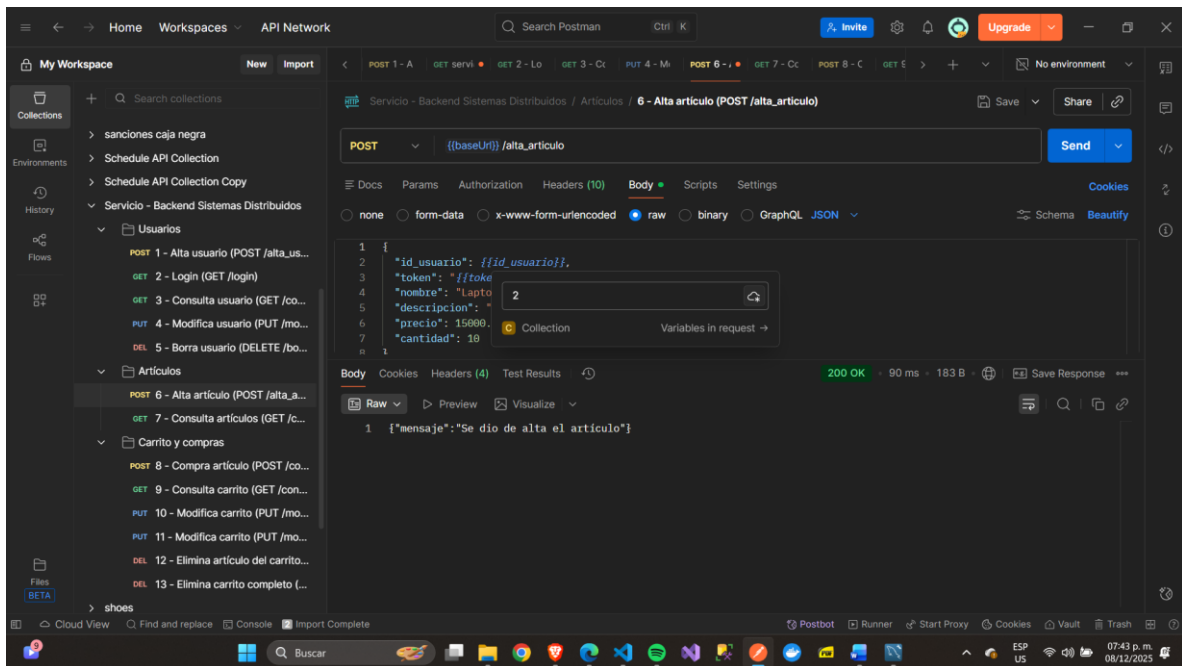
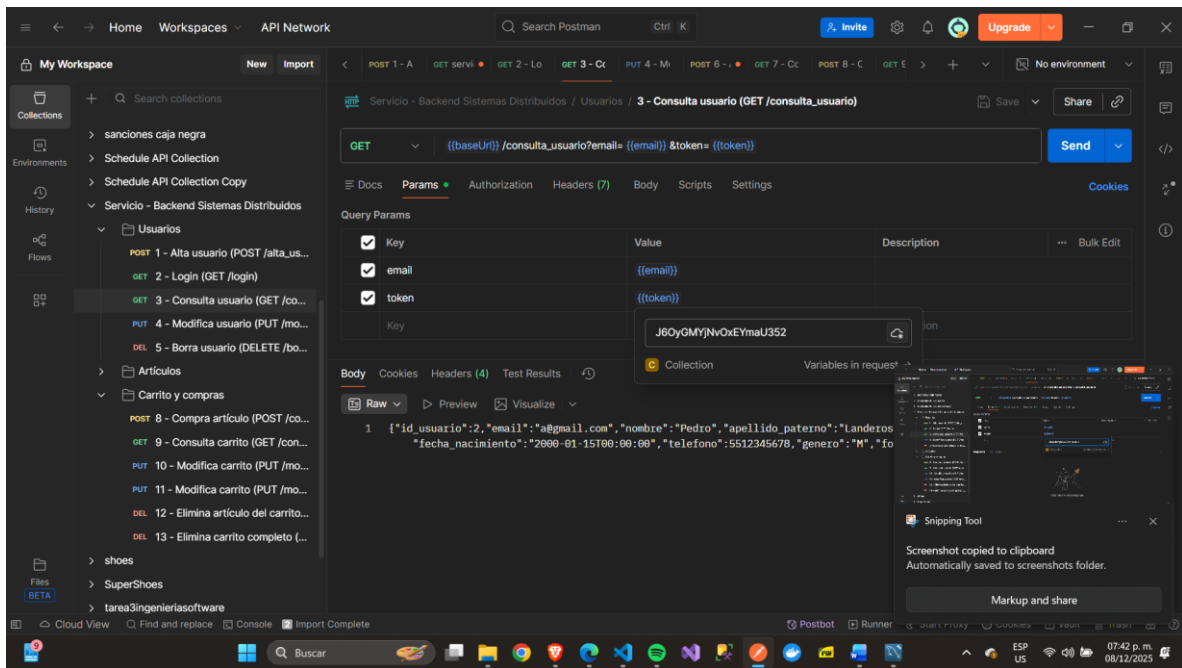
The screenshot shows the Postman interface with the same workspace. The main panel shows a GET request to 'GET 2 - Login' with the URL 'http://localhost:7071/api/login?email={{email}}&password={{password}}'. The request parameters are defined in the 'Params' tab. The response is a 200 OK status with a token: 'eyJ0eXAiOiJKd1VudXNlbnR1eXUyMzUyIn0='.

```
GET http://localhost:7071/api/login?email={{email}}&password={{password}}
```

Key	Value	Description
email	{{email}}	
password	{{password}}	

Response: 200 OK · 108 ms · 174 B

```
{
  "token": "eyJ0eXAiOiJKd1VudXNlbnR1eXUyMzUyIn0="
}
```



Postman interface showing a GET request to `consultar_articulos` endpoint. The request is successful (200 OK) and returns a JSON response.

Request:

- Method: GET
- URL: `{{baseUri}}/consultar_articulos?palabra=laptop&id_usuario={{id_usuario}}&token={{token}}`

Query Params:

Key	Value	Description
palabra	laptop	
id_usuario	{{id_usuario}}	
token	{{token}}	

Response:

```
1 [{"id_articulo":1,"nombre":"Laptop Gamer 2","descripcion":"Laptop potente para juegos","precio":15000.50,"cantidad":10,"foto":null}]
```

Postman interface showing a POST request to `comprar_articulo` endpoint. The request is successful (200 OK) and returns a JSON response.

Request:

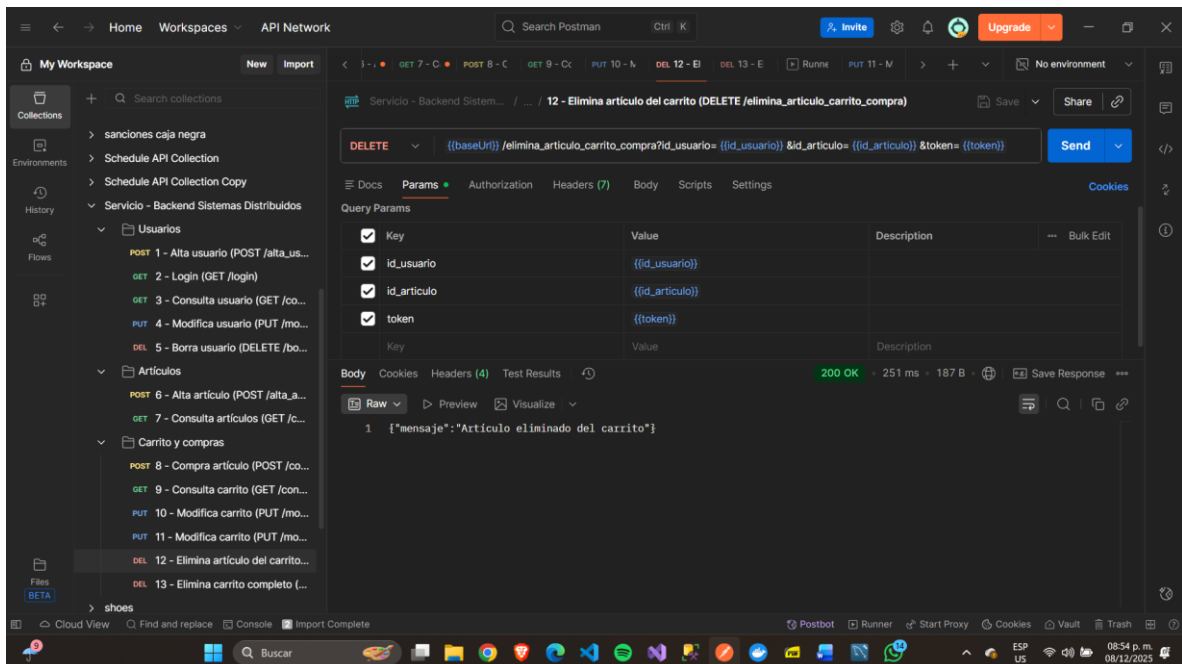
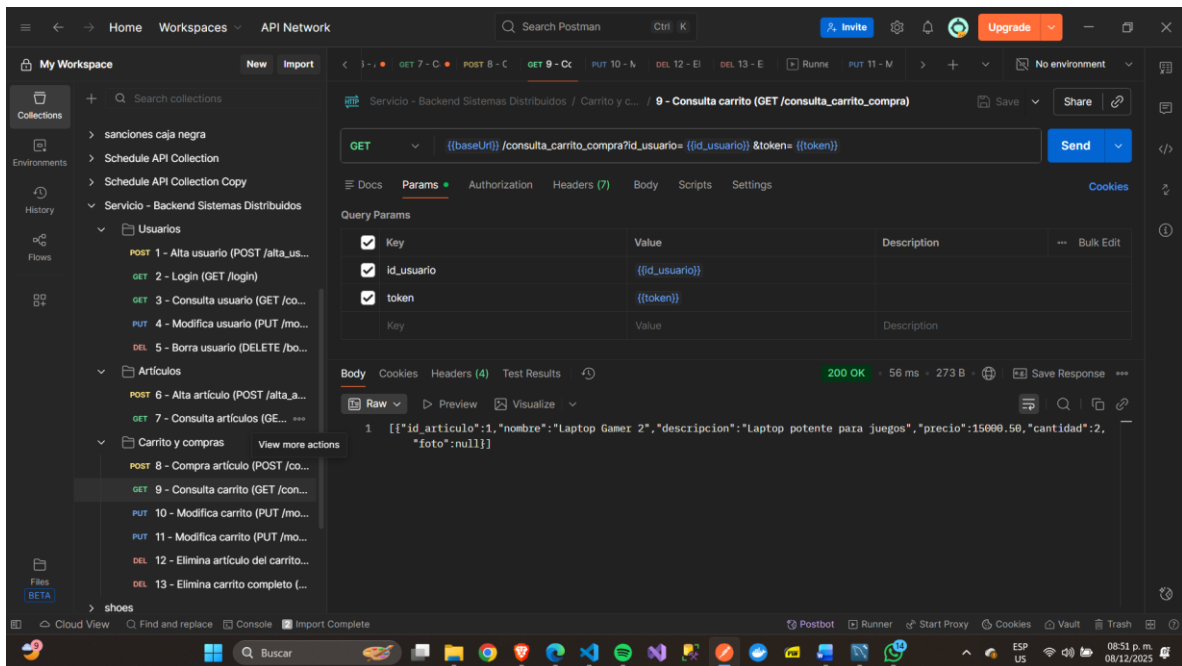
- Method: POST
- URL: `{{baseUri}}/comprar_articulo`

Body (raw):

```
1 {
2   "id_usuario": {{id_usuario}},
3   "token": "{{token}}",
4   "id_articulo": {{id_articulo}},
5   "cantidad": 2
6 }
```

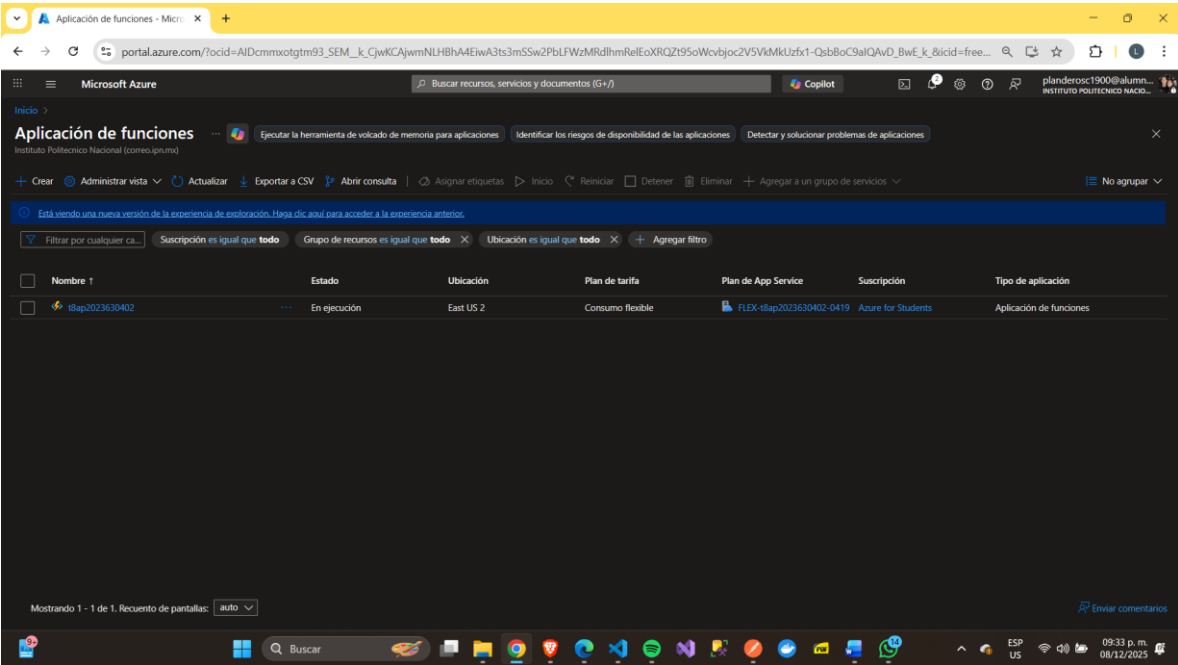
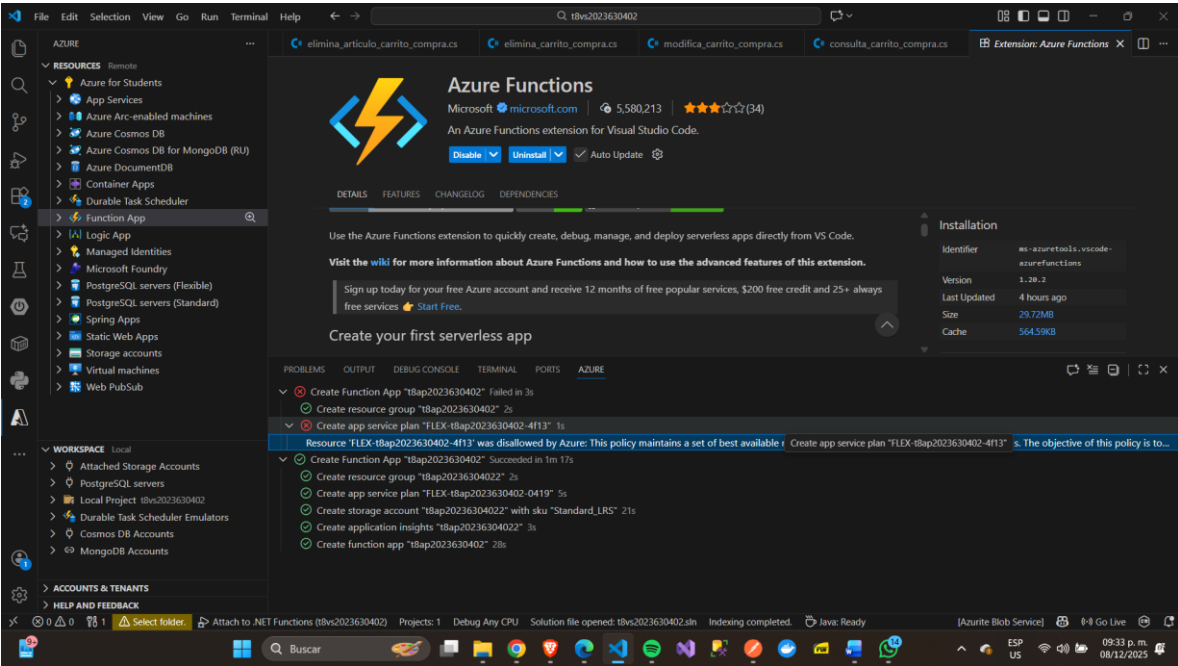
Response:

```
1 {"mensaje":"Articulo agregado al carrito"}
```

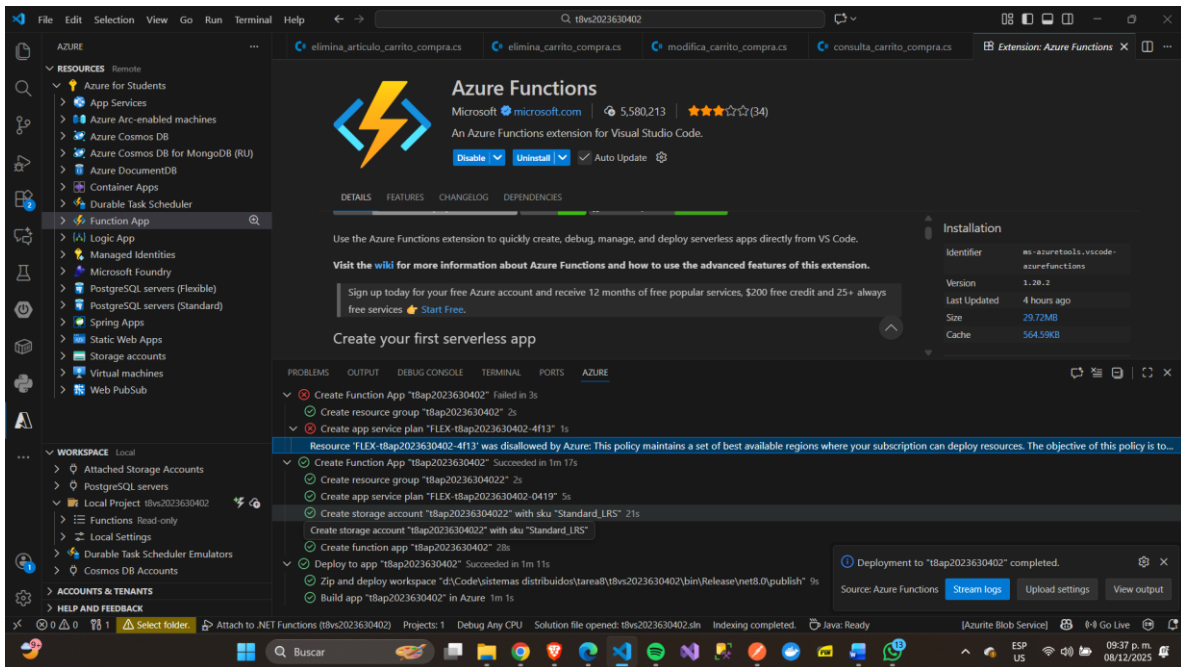
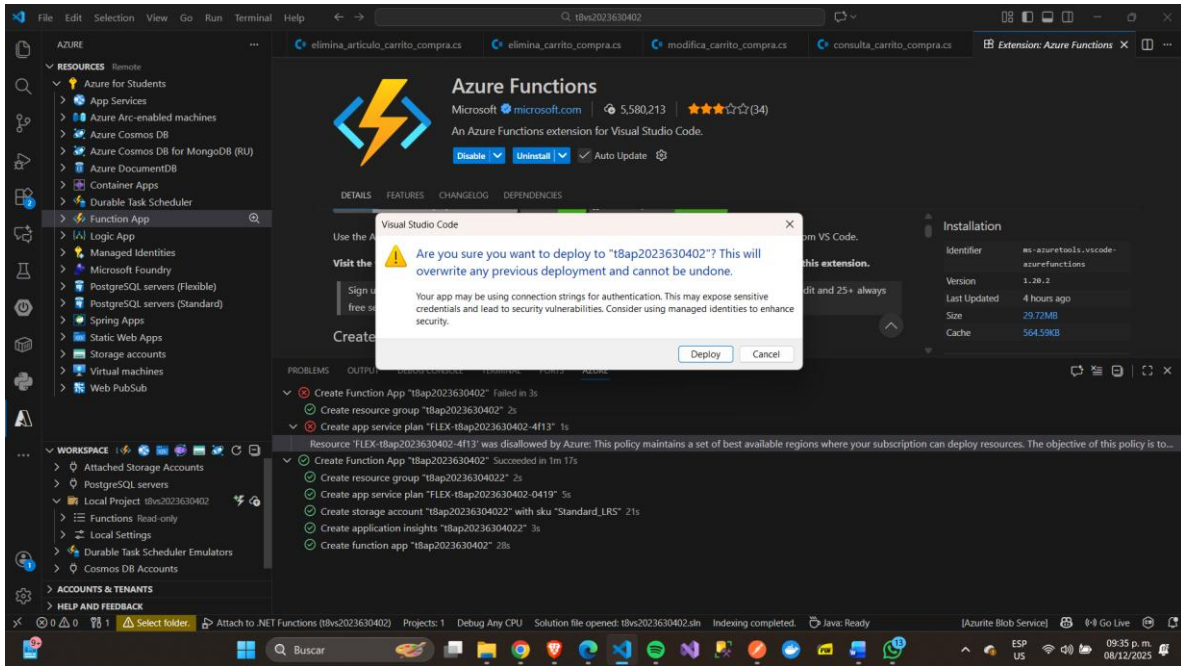


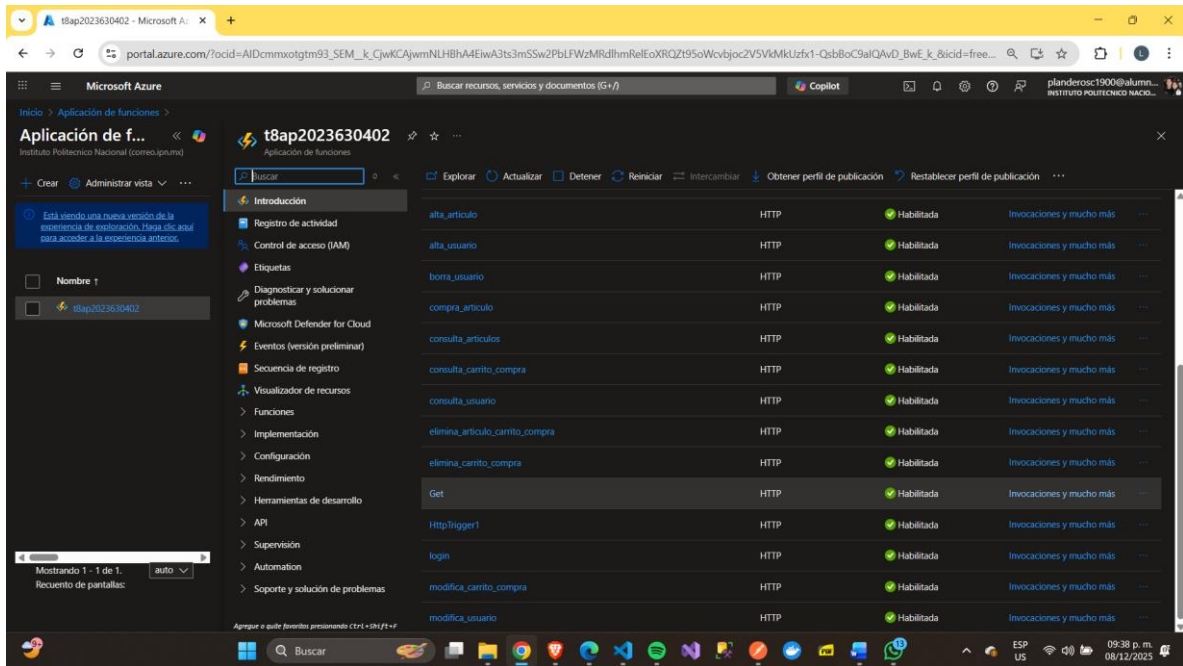
Creacion de la function app

Se hizo la creación desde vs, con .net 8, el nombre indicado, región us east 2.

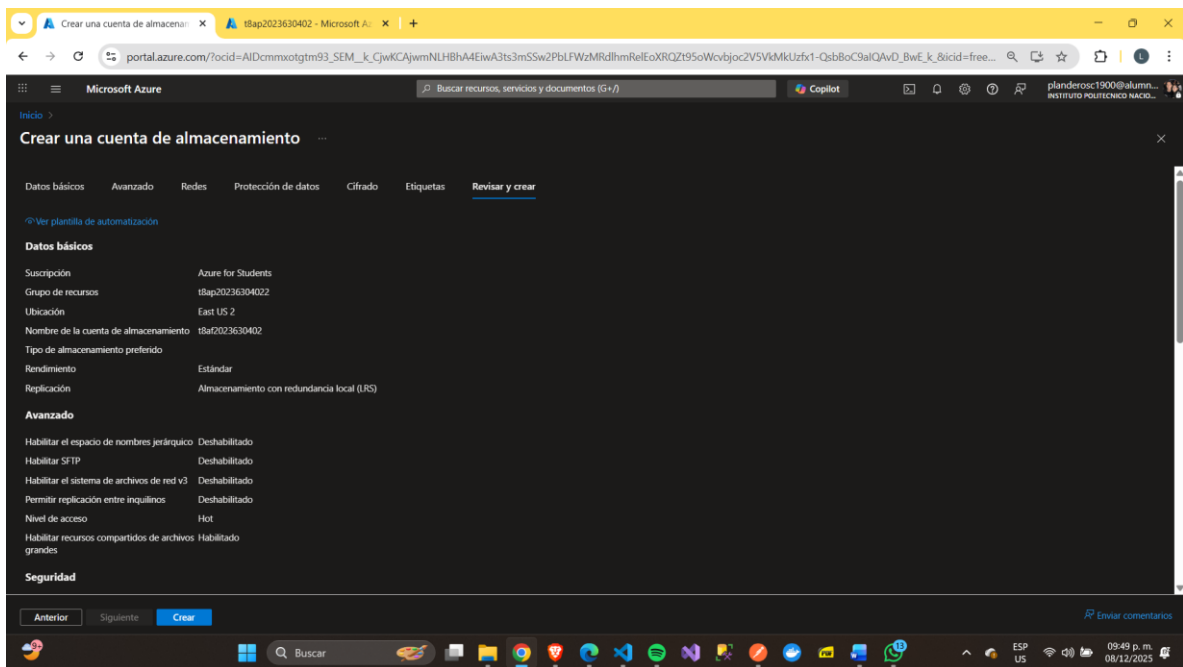


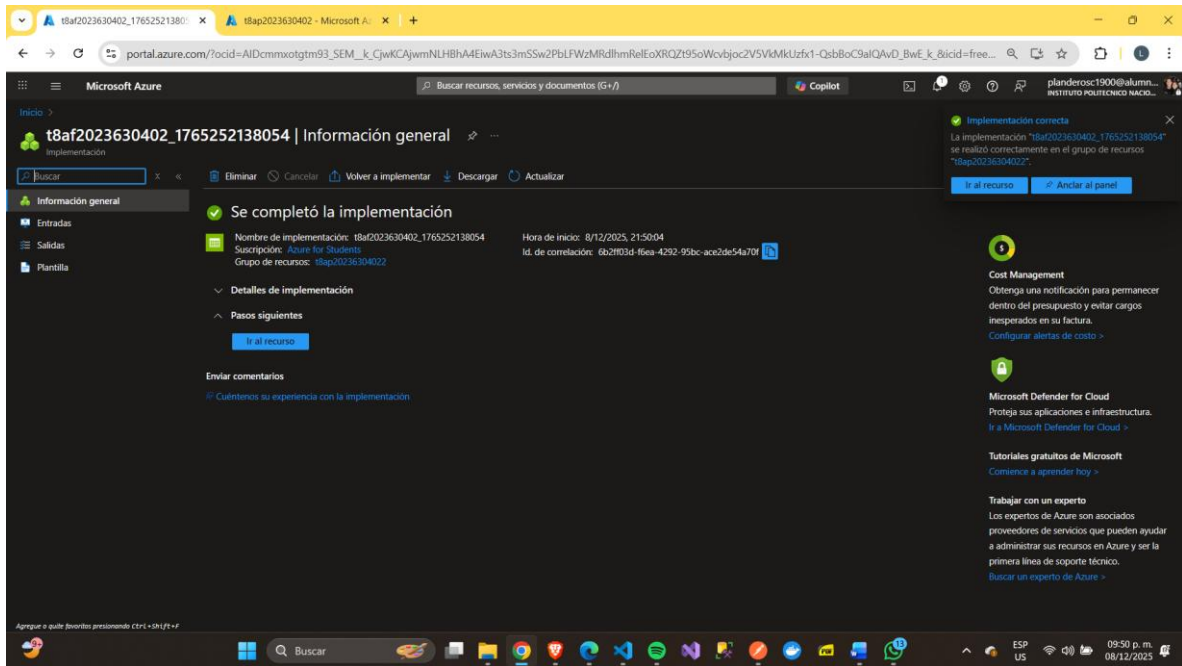
Deploy de la app





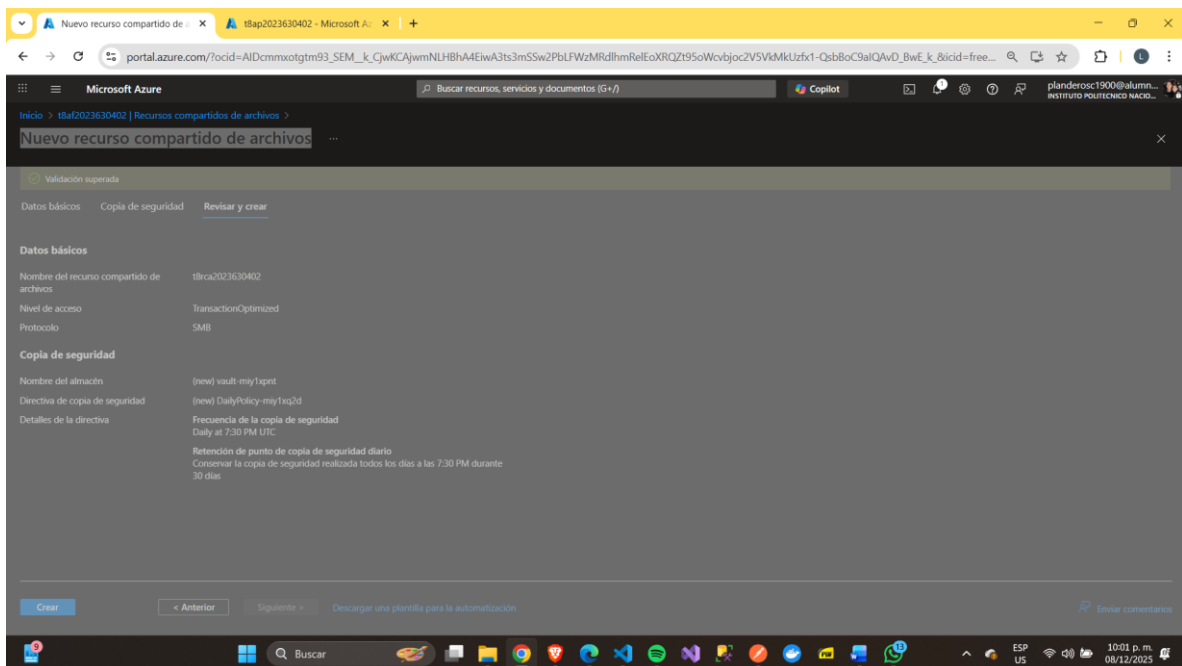
Cuenta de almacenamiento



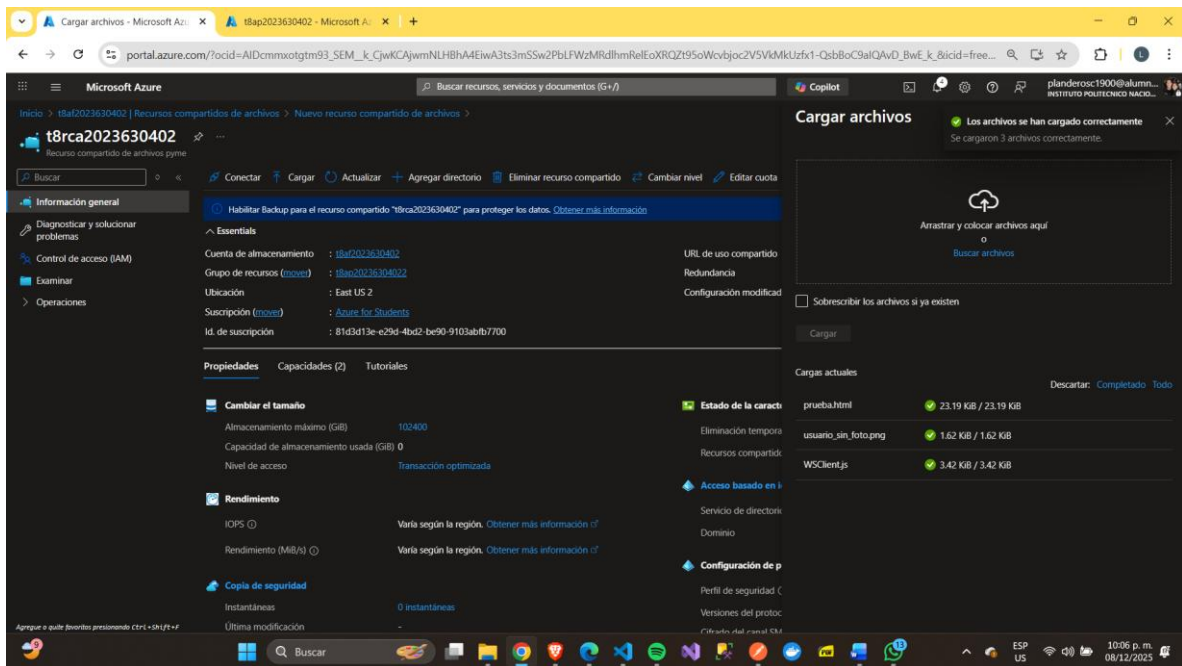
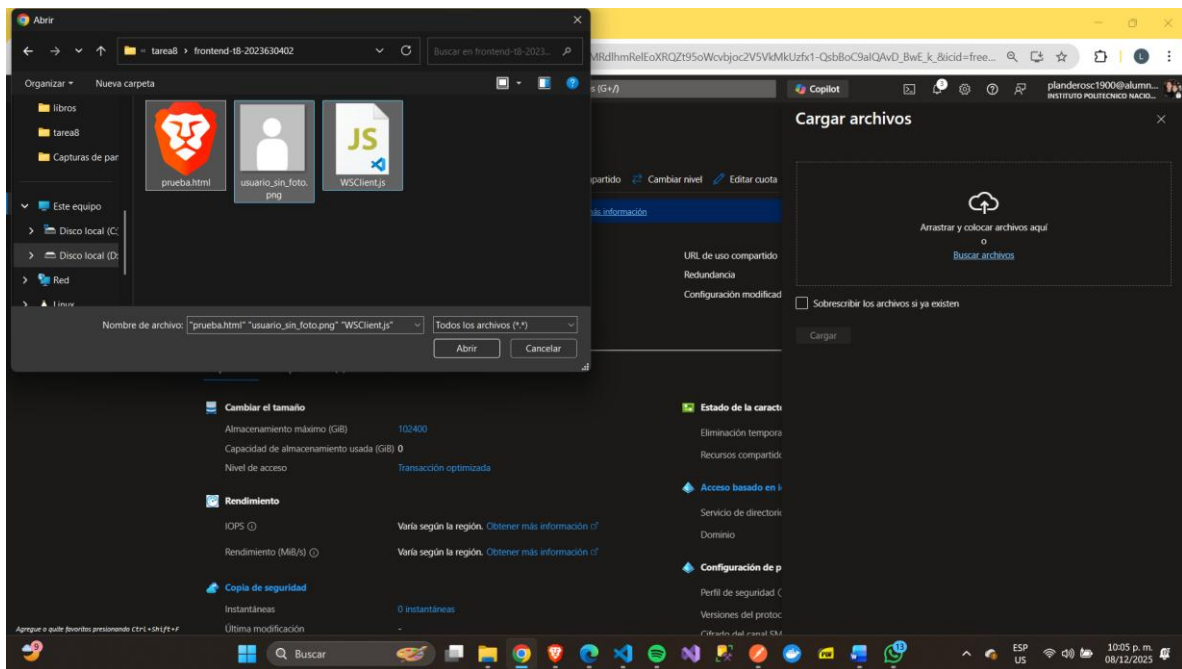


Recurso compartido de archivos

Creacion



Subida de archivos



Montaje de almacenamiento en la aplicación de funciones

The screenshot displays the Microsoft Azure portal interface. The main page shows the 'Asignaciones de ruta de acceso' (Access path assignments) section for the function app 't8ap2023630402'. A table lists the storage mounts, with one entry selected:

Nombre	Ruta de acceso de mon...	Tipo	Nombre de cuenta	Nombre del rec.
t8pm2023630402	/t8pm2023630402	AzureFiles	t8ra2023630402	t8ra2023630402

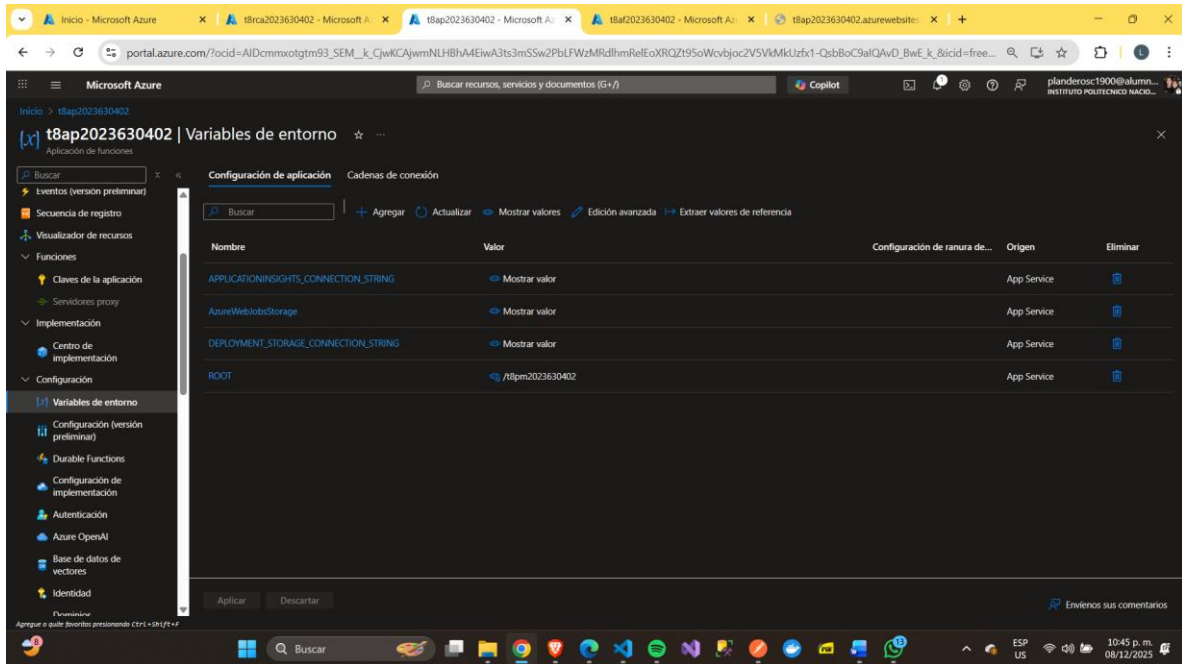
On the right, the 'Agregar montaje de almacenamiento' (Add storage mount) dialog is open. It contains the following configuration details:

- Nombre:** t8pm2023630402
- Opciones de configuración:** Básico (selected), Avanzado
- Nombre de cuenta:** t8ra2023630402
- Tipo de almacenamiento:** Azure Files (selected), Azure Blob (solo lectura)
- Protocolo:** SMB (selected), NFS
- Contenedor de almacenamiento:** t8ra2023630402
- Ruta de acceso de montaje:** /t8pm2023630402
- Configuración de tanura de implementación:** (unchecked)

The dialog also includes a warning message: 'La cuenta de almacenamiento de Azure y la aplicación web deben formar parte de la misma topología de red virtual, si se aplica. Más información'.

Configuración de variables de entorno en aplicación de funciones

Root

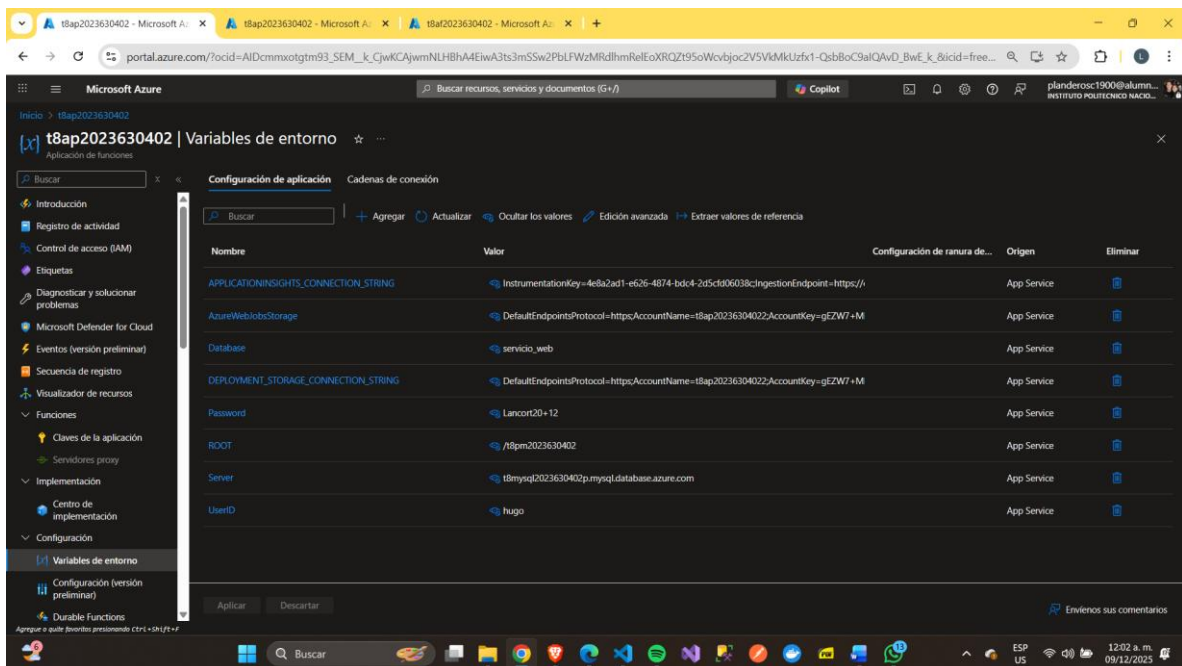


The screenshot shows the Microsoft Azure portal interface for the application 't8ap2023630402'. The 'Variables de entorno' (Environment Variables) section is selected in the left-hand navigation pane. The 'Configuración de aplicación' (Application Configuration) tab is active, displaying a table of environment variables. The table has columns for 'Nombre' (Name), 'Valor' (Value), 'Configuración de ranura de...' (Slot Configuration), 'Origen' (Source), and 'Eliminar' (Delete). The variables listed are:

Nombre	Valor	Configuración de ranura de...	Origen	Eliminar
APPLICATIONINSIGHTS_CONNECTION_STRING	Mostrar valor		App Service	
AzureWebJobsStorage	Mostrar valor		App Service	
DEPLOYMENT_STORAGE_CONNECTION_STRING	Mostrar valor		App Service	
ROOT	/t8pm2023630402		App Service	

Buttons for 'Agregar' (Add), 'Actualizar' (Update), 'Mostrar valores' (Show values), 'Edición avanzada' (Advanced editing), and 'Extraer valores de referencia' (Extract reference values) are visible above the table. At the bottom of the table are 'Aplicar' (Apply) and 'Descartar' (Discard) buttons.

Server



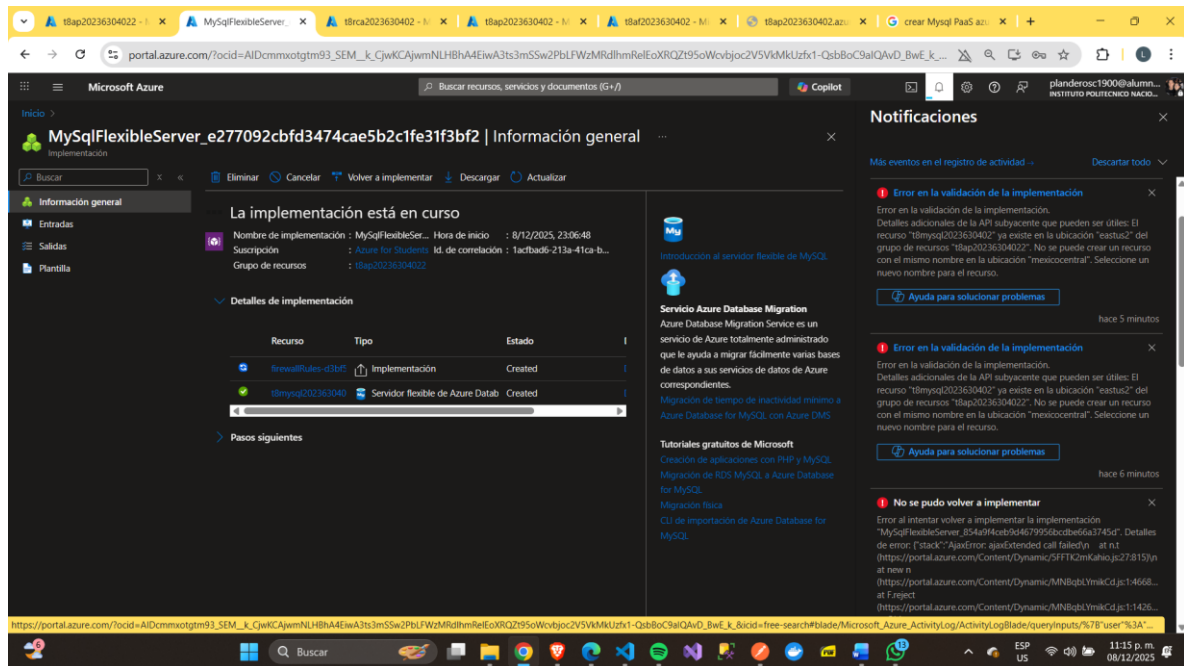
The screenshot shows the Microsoft Azure portal interface for the application 't8ap2023630402'. The 'Variables de entorno' (Environment Variables) section is selected in the left-hand navigation pane. The 'Configuración de aplicación' (Application Configuration) tab is active, displaying a table of environment variables. The table has columns for 'Nombre' (Name), 'Valor' (Value), 'Configuración de ranura de...' (Slot Configuration), 'Origen' (Source), and 'Eliminar' (Delete). The variables listed are:

Nombre	Valor	Configuración de ranura de...	Origen	Eliminar
APPLICATIONINSIGHTS_CONNECTION_STRING	InstrumentationKey=4e8a2ad1-e626-4874-bdc4-2d5c1806038c;ingestionEndpoint=https://		App Service	
AzureWebJobsStorage	DefaultEndpointsProtocol=https;AccountName=t8ap2023630402;AccountKey=gEZW7+M		App Service	
Database	servicio_web		App Service	
DEPLOYMENT_STORAGE_CONNECTION_STRING	DefaultEndpointsProtocol=https;AccountName=t8ap2023630402;AccountKey=gEZW7+M		App Service	
Password	Lancor20+12		App Service	
ROOT	/t8pm2023630402		App Service	
Server	t8mysql2023630402p.mysql.database.azure.com		App Service	
UserID	hugo		App Service	

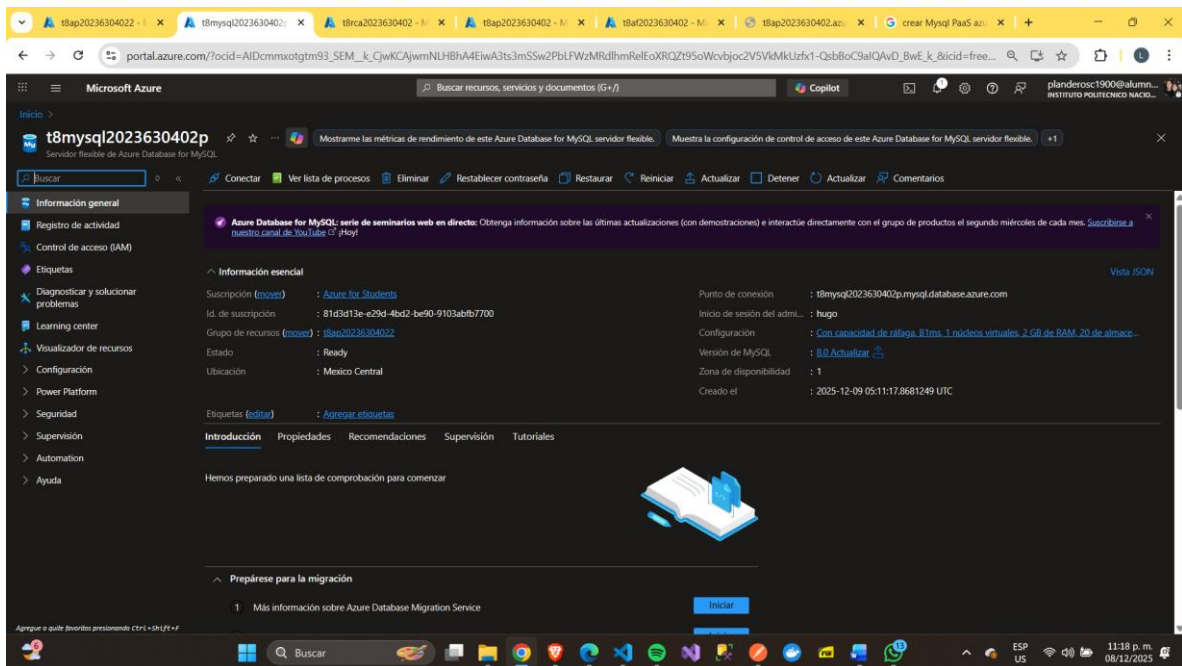
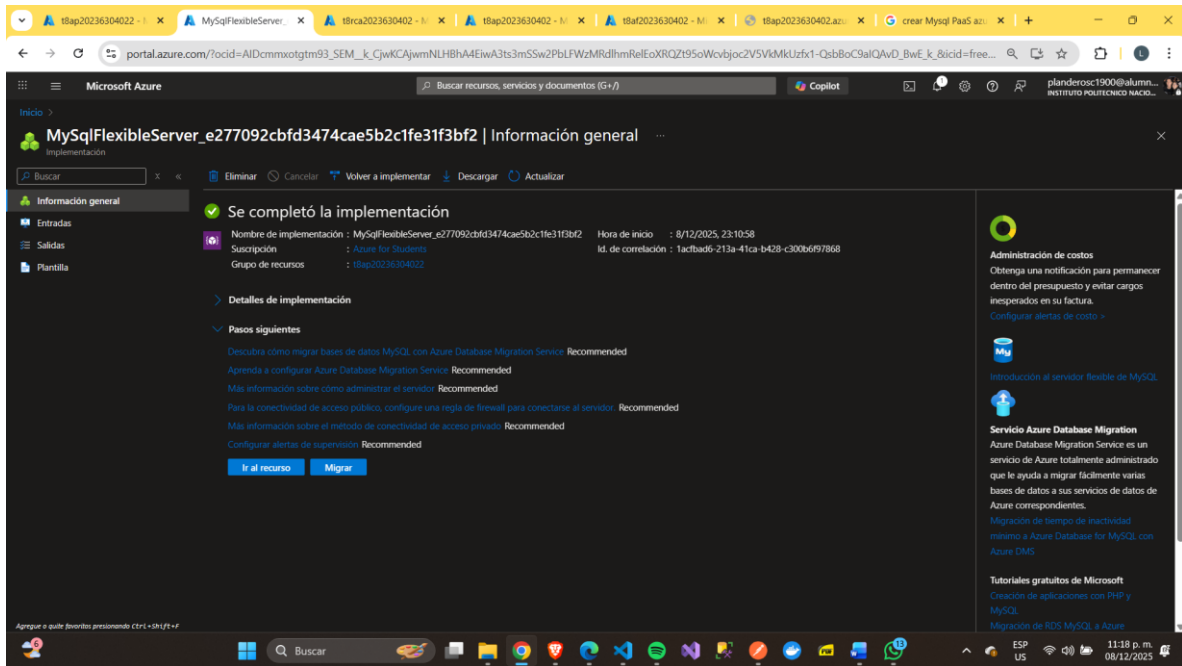
Buttons for 'Agregar' (Add), 'Actualizar' (Update), 'Ocultar los valores' (Hide values), 'Edición avanzada' (Advanced editing), and 'Extraer valores de referencia' (Extract reference values) are visible above the table. At the bottom of the table are 'Aplicar' (Apply) and 'Descartar' (Discard) buttons.

Creacion de base de datos PaaS

Se intento hacer una primera creación y no se pudo



Y cuando se intento volver a usar el mismo nombre marco un error incluso cuando la base de datos ya se había eliminado, por lo que se agrego una p al final del nuevo nombre.



Script para la creación de la base de datos y las tablas en PaaS

-- 1) Crear base de datos

CREATE DATABASE servicio_web;

-- Usar la base de datos

```
USE servicio_web;
```

```
-- 2) Tabla usuarios
```

```
CREATE TABLE usuarios (  
    id_usuario    INT AUTO_INCREMENT PRIMARY KEY,  
    password      VARCHAR(64) NOT NULL,  
    token         VARCHAR(20),  
    email         VARCHAR(100) NOT NULL,  
    nombre        VARCHAR(100) NOT NULL,  
    apellido_paterno VARCHAR(100) NOT NULL,  
    apellido_materno VARCHAR(100),  
    fecha_nacimiento DATETIME NOT NULL,  
    telefono      BIGINT,  
    genero        CHAR(1)  
) ENGINE=InnoDB;
```

```
-- 3) Tabla fotos_usuarios
```

```
CREATE TABLE fotos_usuarios (  
    id_foto  INT AUTO_INCREMENT PRIMARY KEY,  
    foto     LONGBLOB,  
    id_usuario INT NOT NULL  
) ENGINE=InnoDB;
```

```
-- FK de fotos_usuarios → usuarios
```

```
ALTER TABLE fotos_usuarios  
  
ADD CONSTRAINT fk_fotos_usuarios_usuario
```

```
FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario);
```

```
-- Índice único en email
```

```
CREATE UNIQUE INDEX usuarios_1 ON usuarios(email);
```

```
-- 4) Tabla stock (OJO: aquí ya va 'cantidad', no 'existencia')
```

```
CREATE TABLE stock (
```

```
    id_articulo INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre     VARCHAR(100) NOT NULL,
```

```
    descripcion VARCHAR(255) NOT NULL,
```

```
    precio     DECIMAL(10,2) NOT NULL,
```

```
    cantidad   INT NOT NULL,      -- ← ya corregido
```

```
    id_usuario INT NOT NULL,
```

```
    CONSTRAINT fk_stock_usuario
```

```
        FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
```

```
) ENGINE=InnoDB;
```

```
-- 5) Tabla fotos_articulos
```

```
CREATE TABLE fotos_articulos (
```

```
    id_foto    INT AUTO_INCREMENT PRIMARY KEY,
```

```
    foto       LONGBLOB NOT NULL,
```

```
    id_articulo INT NOT NULL,
```

```
    CONSTRAINT fk_foto_articulo
```

```
        FOREIGN KEY (id_articulo) REFERENCES stock(id_articulo)
```

```
) ENGINE=InnoDB;
```

-- 6) Tabla carrito_compra

CREATE TABLE carrito_compra (

id_carrito INT AUTO_INCREMENT PRIMARY KEY,

id_usuario INT NOT NULL,

id_articulo INT NOT NULL,

cantidad INT NOT NULL,

CONSTRAINT fk_carrito_usuario

FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),

CONSTRAINT fk_carrito_articulo

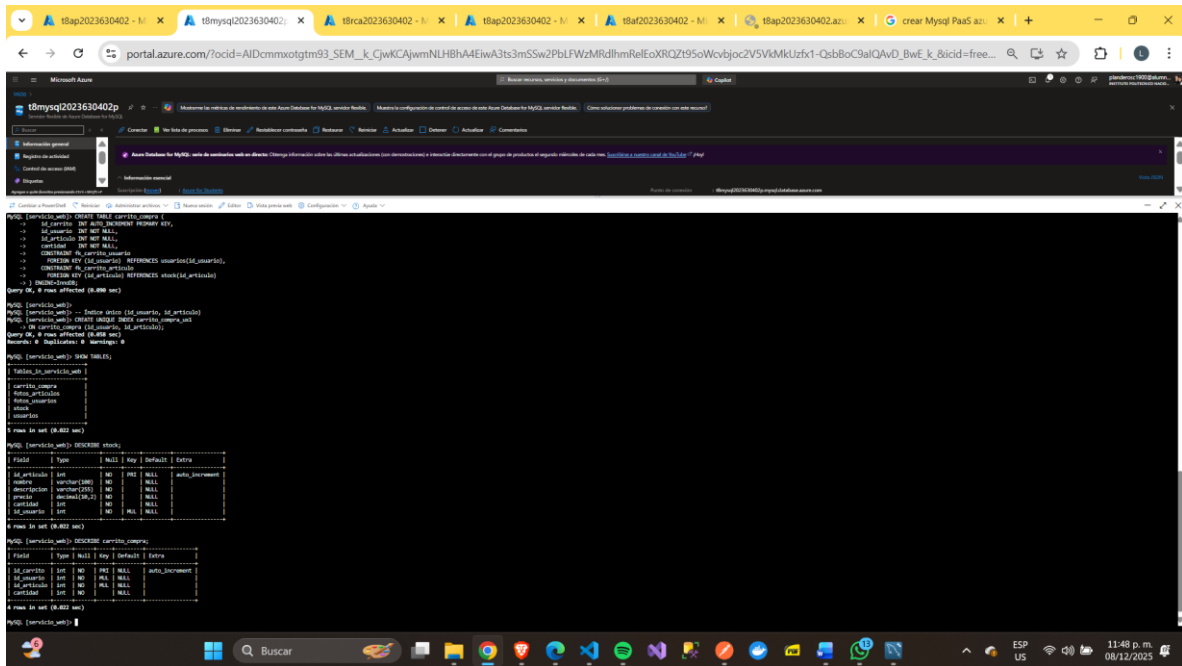
FOREIGN KEY (id_articulo) REFERENCES stock(id_articulo)

) ENGINE=InnoDB;

-- Índice único (id_usuario, id_articulo)

CREATE UNIQUE INDEX carrito_compra_ux1

ON carrito_compra (id_usuario, id_articulo);



Pruebas de la app desde el celular

11:43



Acceso al sistema

Email

Password

Aceptar

Registro de usuario

2023630402.azurewebsites.net



11:43



Registro de usuario

Email *

test2@gmail.com

Contraseña *

••••

Nombre *

Pedro

Apellido paterno *

Landeros

Apellido materno

Cortes

Se registró el usuario

Close

Genero

Masculino



Choose Files

no files selected

Registrar usuario

Cancelar

2023630402.azurewebsites.net



11:44



Perfil del usuario

Captura de artículo

Compra de artículos

Eliminar perfil

Salir

2023630402.azurewebsites.net



11:44



Perfil del usuario

Email *

test2@gmail.com

Contraseña *

Nombre *

Pedro

Apellido paterno *

Landeros

Apellido materno

Cortes

Fecha de nacimiento *

1 Dec 2025 at 11:43 p.m.

Teléfono

11223344

Genero

Masculino



Choose Files

no files selected

Quitar foto

Guardar cambios

Regresar

2023630402.azurewebsites.net



11:44



Captura de artículo

Nombre *

Artículo test af

Descripción *

Quwuqu

Precio *

12

Existencia *

123

Se dio de alta el artículo

Close



Choose Files



2AF4F4C2-08...AF049.jpeg

Guardar artículo

Regresar

2023630402.azurewebsites.net



11:44



Compra de artículos

Palabra clave

Buscar

Carrito de compra



Artículo test af

Quwuqu

Precio: \$12

Cantidad:

Compra

Regresar al menú

2023630402.azurewebsites.net



11:45



Compra de artículos

Palabra clave

Buscar

Carrito de compra



Artículo test af

Quwuqu

Precio: \$12

Cantidad:

Compra

Regresar al menú

2023630402.azurewebsites.net



11:45



Compra de artículos

Palabra clave

Buscar

Carrito de compra

Se agregó el artículo al carrito

Close

Artículo test ar

Quwuqu

Precio: \$12

Cantidad:

Compra

Regresar al menú

2023630402.azurewebsites.net



11:45



Artículos en el carrito



Artículo test af

Precio: \$12

Cantidad: 4

Subtotal: \$48.00

[Eliminar artículo](#)

Total: \$48.00

[Eliminar carrito](#)

[Seguir comprando](#)

[Regresar al menú](#)

2023630402.azurewebsites.net



11:45



Artículos en el carrito



Artículo test af

Precio: \$12

Cantidad: 4

Subtotal: \$48.00

¿Deseas eliminar este artículo del carrito?

Cancel OK

Seguir comprando

Regresar al menú

2023630402.azurewebsites.net



11:45



Artículos en el carrito



Artículo test af

Precio: \$12

Cantidad: 4

Subtotal: \$48.00

Se eliminó el artículo del carrito

Close

Seguir comprando

Regresar al menú

2023630402.azurewebsites.net



11:45



Compra de artículos

Palabra clave

Buscar

Carrito de compra



Artículo test af

Quwuqu

Precio: \$12

Cantidad:

Compra

Regresar al menú

2023630402.azurewebsites.net



11:45



Artículos en el carrito



Artículo test af

Precio: \$12

Cantidad: 3

Subtotal: \$36.00

¿Deseas eliminar todos los artículos del carrito?

Cancel OK

Seguir comprando

Regresar al menú

2023630402.azurewebsites.net



11:45



Artículos en el carrito



Artículo test af

Precio: \$12

Cantidad: 3

Subtotal: \$36.00

Se eliminó el carrito de compra

[Close](#)

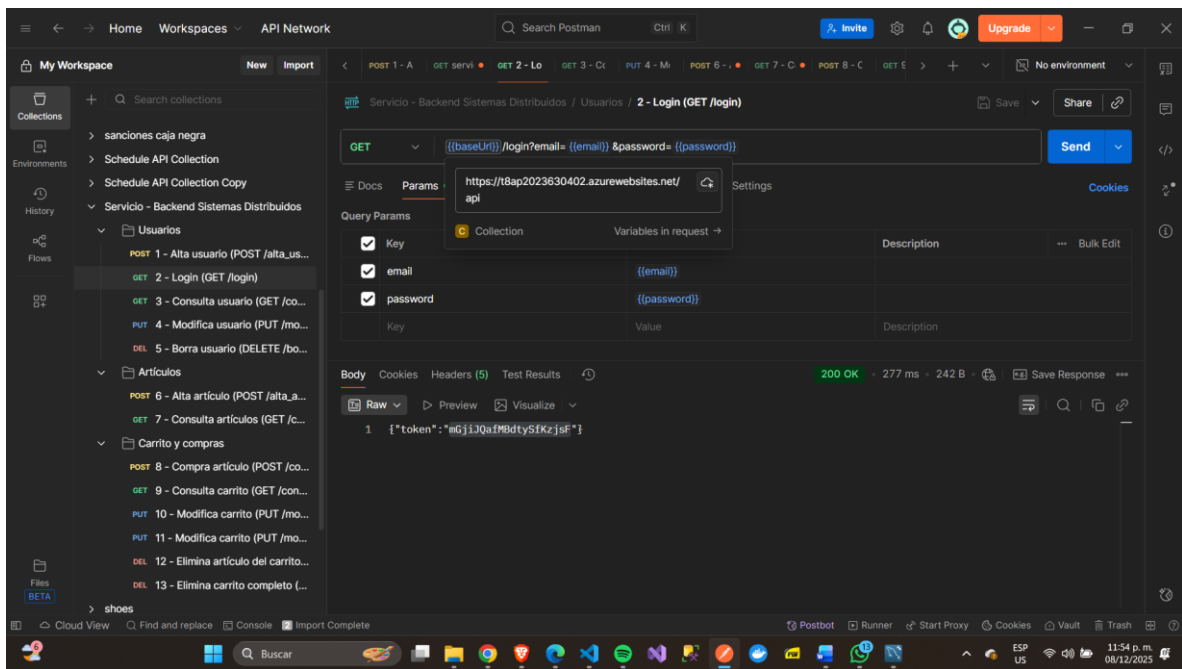
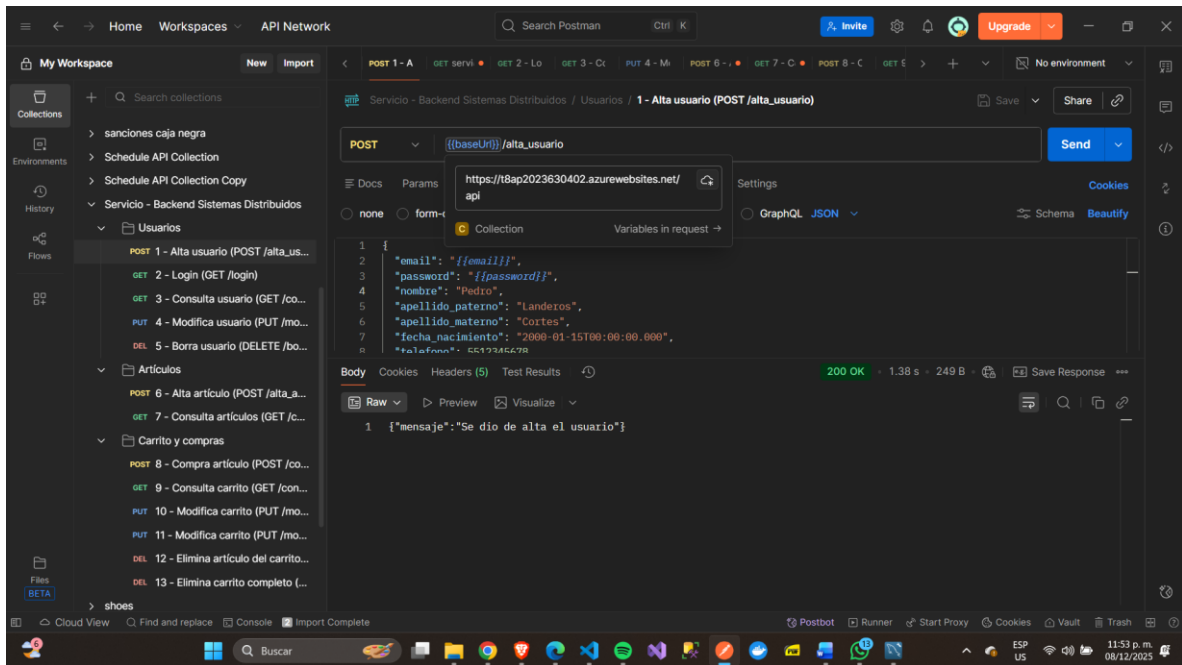
[Seguir comprando](#)

[Regresar al menú](#)

 2023630402.azurewebsites.net 



Pruebas de la function app desde Postman



Postman interface showing a GET request to `/consulta_usuario?email={{email}}&token={{token}}`. The request is part of a collection named "Servicio - Backend Sistemas Distribuidos / Usuarios". The response is a JSON object:

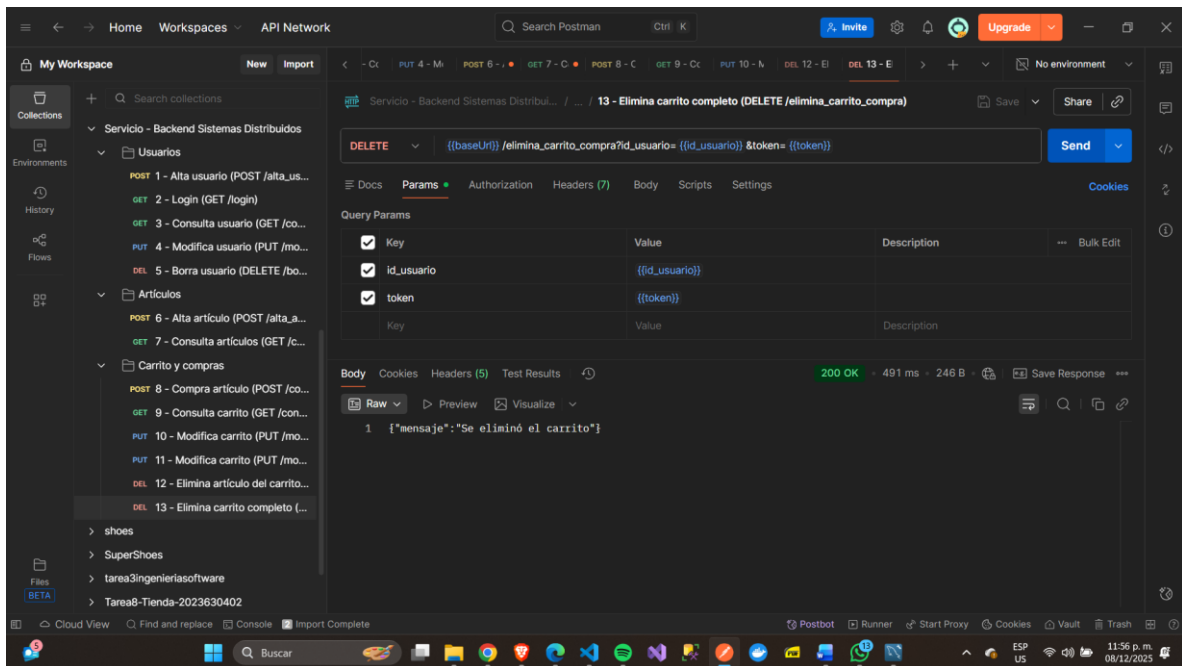
```
1 [{"id_usuario": 3, "email": "atgmail.com", "nombre": "Pedro", "apellido_paterno": "Landeros", "apellido_materno": "Cortes", "fecha_nacimiento": "2000-01-15T00:00:00", "telefono": "5512345678", "genero": "M", "foto": null}]
```

Postman interface showing a POST request to `/alta_articulo`. The request is part of a collection named "Servicio - Backend Sistemas Distribuidos / Articulos". The request body is a JSON object:

```
1 {
2   "id_usuario": {{id_usuario}},
3   "token": "{{token}}",
4   "nombre": "Lapto",
5   "descripcion": "
6   "precio": 15000,
7   "cantidad": 10
8 }
```

The response is a 200 OK status with a message:

```
1 {"mensaje": "Se dio de alta el articulo"}
```

Conclusiones

El desarrollo de este prototipo de sistema de comercio electrónico basado en una arquitectura serverless me permitió comprender y aplicar de manera práctica diferentes conceptos fundamentales relacionados con los servicios en la nube. A lo largo del proyecto pude implementar un backend funcional utilizando Azure Functions en .NET 8, integrado con una base de datos MySQL desplegada como servidor flexible PaaS. También configuré el acceso seguro mediante TLS, reglas de firewall y variables de entorno, lo cual me ayudó a entender mejor cómo se establece la comunicación entre servicios distribuidos.

Además, implementé cada uno de los requerimientos funcionales solicitados: el registro y autenticación de usuarios, la captura y consulta de artículos, la compra mediante actualización transaccional de stock y la administración completa del carrito de compra. Para lograrlo, trabajé con transacciones, validaciones y control de concurrencia, lo que me permitió reforzar mi habilidad para manejar la integridad de datos en un sistema distribuido. De igual manera, adapté el front-end para interactuar correctamente con las funciones, y pude desplegarlo en Azure Files, verificando la correcta entrega de contenido en la nube.

El proceso de despliegue me llevó también a crear y configurar recursos en Azure, como la Function App, el almacenamiento, el File Share y el servidor MySQL, así

como su conexión mediante credenciales y cadenas de conexión. Este flujo completo me ayudó a comprender cómo diferentes servicios cloud se integran para formar una solución funcional, escalable y sin necesidad de administrar servidores directamente.

Finalmente, las pruebas realizadas tanto desde herramientas locales como desde un dispositivo móvil confirmaron que el sistema cumple con los requerimientos del back-end y del front-end. En general, este proyecto representó una experiencia completa que me permitió unir desarrollo, configuración de infraestructura y despliegue en la nube, fortaleciendo mi entendimiento sobre cómo se construyen aplicaciones modernas basadas en arquitectura serverless.

Link al repositorio con el código completo:

<https://github.com/PedroLanderos/SistemasDistribuidos/tree/main/tarea8>